

Semantic Streams Augmentation

2026 Sebastián Samaruga. <https://sebxama.blogspot.com>

This work is licensed under a [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License](#).

Summary:

This document describes a semantic RDF based data integration and intelligence framework by means of Aggregation, Alignment and Activation of ontologies.

Base Model

```
class Resource
+ ID : URI
+ primeID : BigInteger
+ occurrences : Occurrence[]

class Occurrence extends Resource
+ player : Resource
+ context : Resource
+ role : Resource (Kind?)

class Statement extends Resource
+ context : Context
+ subject : Subject
+ predicate : Predicate
+ object : Object

class Context extends Occurrence
+ role : ContextKind

class Subject extends Occurrence
+ role : SubjectKind

class Predicate extends Occurrence
+ role : PredicateKind

class Object extends Occurrence
+ role : ObjectKind
```

```

interface Kind<PlayerType extends Occurrence,
    AttributeType extends Occurrence,
    ValueType extends Occurrence>

class ContextKind extends Context implements Kind<Context, Object, Subject>

class SubjectKind extends Subject implements Kind<Subject, Predicate, Object>

class PredicateKind extends Predicate implements Kind<Predicate, Subject,
Object>

class ObjectKind extends Object implements Kind<Object, Predicate, Subject>

```

Resources

Everything is a Resource identified by an URI, assigned a Prime Number ID, keeping track of its Occurrences and thus enabling the calculation of their CPPE (Prime Number Contextual Embedding) by means of the product of its Prime Number ID by the Prime Number IDs of the other Resources in their co-Occurrences.

Occurrences

Participation of a Resource in an Occurrence in a given Role (Kind) in a given Context (Statement, Kind).

Statement Occurrences

Statements are Resources whose occurrences are their CSPOs in a given Role (Kind?)

CSPO Occurrences

Context, Subject, Predicate, Object as Resource(s). SPO occurring Statement Contexts where they are in C, S, P, O Roles.

CSPOs instances as wrapped arbitrary Resource Occurrence Context

Kinds Occurrences

As Context, Subject, Predicate, Object. Occurrences: Kind Functional Definition Statement Contexts (occurring Statement Contexts)

Kinds as Resources: Kinds occurring in Statements in C, S, P, O Roles

Statements

Context, Subject, Predicate and Object (CSPO) tuples.

Context, Subject, Predicate, Object (CSPO) Statement Occurrences have a role (Kind) in their given Statement Occurrence Context.

Kinds: ContextKind, SubjectKind, PredicateKind, ObjectKind (CK, SK, PK, OK) are themselves CSPOs respectively and can participate in a Statement Context as Occurrences.

Types of Statements:

- SPO Statements
- SPO / Kinds Statements (recursive Kinds)
- Kinds only Statements (recursive Kinds)

This is further leveraged in the Augmentation Layers Domain Model definitions section.

Contexts, Subjects, Predicates, Objects

The role of a Resource Occurrence in a Statement is typed by its Statement Occurrence position. A Resource can occur in more than one role or occurrence type (CSPO) in different Statements. The type of the role (Kind) of the Resource Occurrence is determined by the type of the position it plays in a given Statement.

Context, Subject, Predicate, Object Kinds

Aggregation of PlayerType(s) Resource Occurrences (SPOs) by common AttributeType(s) and then by common ValueTypes(s). Parameterized interface for each Kind type (CK, SK, PK, OK).

Kinds can be aggregated from raw CSPO Occurrences from any combination of C, S, P, O / CK, SK, PK, OK Statement Occurrence types. This allows for recursive Kinds definitions (folding). This is further leveraged in the Augmentation Layers Domain Model definitions section.

Kinds are a kind of “type inference”: if a Subject shares common attributes with another Subject, it is of the same Kind “type”. If their common attributes share common values, it is of the same Kind “state”.

Root Kinds are the Kinds corresponding to the CSPO Occurrences Statements positions. They have no attributes nor values, they just tag an Occurrence as being of an CSPO Occurrence position in a Statement.

Kinds hierarchies: basic inheritance, if a Subject attribute set is the superset of another Subject attribute set, it can be regarded as an “extension” of the first Subject. Leverage this with other inference mechanisms (FCA, for example).

Kinds ordering: a Subject “extended” by another Subject attributes set can be regarded as being “before” the first Subject. This assumes the existence of a super type is necessary for the existence of a sub type.

Kinds are represented and serialized by Statements whose Statement Contexts are the Kind instance itself (Kind reified Context Resource Occurrence), and their Subjects, Predicates and Objects plays the PlayerType, AttributeType and ValueType of the Kind definition. Thus, the definition of a Kind is the set of Statements of its corresponding Kind as Resource Occurrence of their Statement Contexts.

Kinds as Functional Definition Statements

Kind Statements can be regarded as the definition of a Function by extension. Their Statement Contexts Occurrence Resources are the Function name or signature. Their PlayerType(s) instances group their mappings by “context” and their AttributeType(s) and ValueType(s) instances are the Function domain / range corresponding value pairs. Subject

- CK: Context (Contexts), Domain (Subject), Range (Predicates):
aGivenContextKind(C, S): P
- SK: Context (Subjects), Domain (Predicates), Range (Objects):
aGivenSubjectKind(S, P): O
- PK: Context (Predicates), Domain (Subjects), Range (Objects):
aGivenPredicateKind(P, S): O
- OK: Context (Objects), Domain (Predicate), Range (Subject): aGivenObjectKind(O, P): S

Base Model Representations

Base Model class hierarchy is represented by different meta model approaches (kept in sync with each other) for leveraging the most appropriate inference mechanisms.

RDF Model Representation

Raw RDF Statements Triple (Quad) Store. SPARQL Endpoint.

Sets Based Model Representation

Contexts Set (Subjects Set, Predicates Set and Objects Set union).

Subjects Set.

Predicates Set.

Objects Set.

Context Kinds Set (Subject Kinds Set, Predicate Kinds Set, Object Kinds Set union).

Subject Kinds Set (Predicates Set intersection with Objects Set).

Predicate Kinds Set (Subjects Set intersection with Objects Set).

Object Kinds Set (Subjects Set intersection with Predicates Set).

ISO TMRM Model Representation

Underlying common metamodel.

FCA Context Lattices Model Representation

XML Model Representation

TreeGraph for nested Statements structures Stream Processing. Pipeline messages format.

Execution Model: Augmentation Agents Pipeline

A layered Data / Information / Knowledge approach which is intended to consume raw RDF data serialized or obtained by data sources connectors which shall provide synchronization mechanisms for later integration of processed and aggregated Data (Aggregation Layer), inferred matching, completion and dimensional arrangement of Information (Alignment Layer) and activated interactions from API behavior exposure of inferred contexts (Knowledge).

Processing of input Data into activated Knowledge through the Alignment phase is performed by means of stream processing of input Statements by each of the Augmentation Pipeline Layers which performs merging of Statements of each layer Statements into the corresponding already known Statements of those layers and “folding” (aggregating) merge result Statements into the next layer Statement type in the hierarchy and then merging again this next layer acquired knowledge with already known Statements of the corresponding layer until reaching the top (bottom) of the layers hierarchy.

The same process occurs “backwards”, from Statements more “deep” in the aggregation layer hierarchy being merged with already known Statements of those layer and then being “unfolded” (de aggregated) into they composing layers Statements which are merged with the corresponding layer already known Statements and the process repeats until reaching the top of the hierarchy (raw S, P, O Statements).

Homoiconic (codat) approach

Augmentation Agents Pipeline consume Statements produced by any other Agent, including themselves, until reaching primitives. Blackboard messaging pattern, Augmentation Pipeline Layers Agents roles relative to the augmentation steps they perform and not by their position or order of invocation.

Each Layer Statement type builds upon upper Layer Statement types. Lower Layer Statements can be decomposed into the upper layers Statements who were the source they were built upon, until reaching top SPO Data Layer Statements. Also, lower layer statements

types can be aggregated / composed into upper layers statements types. One layer output Statement (SK, PK, OK) is next layer input Statement (S, P, O) and vice versa.

The statements are processed in a streaming fashion merging aligned input and already known statements of the corresponding layers, merging folding and unfolding results in a fan in / fan out approach. Folding and unfolding merge results are published to be further processed and augmented.

Merging input data against previously known data, applying as a Template / Alignment / Transformation is what is meant with “code as data”.

Aggregation, Alignment, Activation Augmentation Pipeline

Messaging Infrastructure

Blackboard style messaging backbone. All services subscribe and publish to the same topic.

Messaging Format: TreeGraph

Raw XML Serialization of Base Model Statements and entities. Layers Statements nested composition (tree like structure). Synchronization to and from an eventual TMRM underlying metamodel.

XSLT for parsing and extracting composed layer statements source layers statements. Statements below in the hierarchy are composed of Statements above them. This XSL Stylesheet transform provides this upper composing layers statements in the context of the layer composed Statement being parsed.

Augmentation: Graph operations performed in transforms results statements:

Downstream layers composition.

Upstream layers decomposition.

On each composition / decomposition step result statements: Merge with Production Statements (Contexts). Compose / decompose merged transform results statements until merge is no op.

Aggregation:

Merge statements obtained with Processing (composing) Statements. FCA Contexts Merge.

Alignment:

CPPE Embeddings. Ontology Matching / Links / Dimensional / Order inference.

Activation:

Further Statements layers composition downstream in the hierarchy.

Augmentation Layers Domain Model

Augmentation Pipeline Layers Statements aggregation levels encoded messages structure.

Each layer encoding structure leverages the representation of Kinds / Roles as functional extension definitions.

Each layer resembles the same aggregation / folding structures, being each layer topmost input statement format (S, P, O) the previous layers structure (SK, PK, OK). For unfolding / de-aggregation of layer statements each layer (S, P, O) is fed into the upper layer (SK, PK, OK) bottom layer.

Subsequent layers Kinds are aggregated from previous layer Statements CSPOs.

Data Layer: Aggregation Agent

DOM Layer: Data / Schema Statements (Roles, Types)

Aggregation phase consumes Data / Instance Statements, produces Property Statements.

Folding: From input Statements through Property Statements by means of Kinds Aggregation.

Unfolding: From aggregated Property Kind Statements into their originating Kind Statements.

Layer Types: Instance / Type / Property

Data / Instance Statements:

Raw CSPO Quad Statements.

(C, S, P, O)

Type Statements:

Layer Kinds.

Each layer calculates its corresponding Kinds from their input CSPO Statements from their above input layer aggregating inputs into their corresponding layer CSPO Kinds (Raw CSPO Statements in this layer).

(CK, S, P, O): (EmploymentContextKind, :Peter, :worksFor, :anEnterprise)

(SK, S, P, O): (EmployeeSubjectKind, :Peter, :employer, :anEnterprise)

(PK, S, P, O): (EmploymentRelationshipPredicateKind, :Peter, :worksFor, :anEnterprise)

(OK, S, P, O): (EmployerObjectKind, :Peter, :employedBy, :anEnterprise)

Property Statements:

Kinds “static schema / class” property definitions. Information Layer Axis CSPO.

(CK, SK, PK, OK): (EmploymentContextKind, EmployeeSubjectKind, EmploymentRelationshipPredicateKind, EmployerObjectKind)

Information Layer: Alignment Agent

Dimensional Layer: State / Transition Statements (Matching, Order)

Alignment Agent (State / Transition Statements. Dimensional Layer). Matching / Order

Alignment phase consumes Aggregation Property Statements, produces Rule Statements.

Folding: From input Statements through Rule Statements by means of Kinds Aggregation.

Unfolding: From aggregated Rule Kind Statements into their originating Kind Statements.

Layer Types: Axis / State / Rule

Axis Statements:

Consumes Data Layer Property CSPO Quad Statements.

(C, S, P, O)

Consumes Data Layer Property Statements as input:

(CK, SK, PK, OK): (EmploymentContextKind, EmployeeSubjectKind, EmploymentRelationshipPredicateKind, EmployerObjectKind) is an example of the source (C, S, P, O) of this layer.

State Statements:

Layer Kinds.

Each layer calculates its corresponding Kinds from their input CSPO Statements from their above input layer aggregating inputs into their corresponding layer CSPO Kinds (Axis CSPO Statements in this layer).

(CK, S, P, O): (AggregatedEmploymentContextKind, EmployeeSubjectKind, EmploymentRelationshipPredicateKind, EmployerObjectKind)

(SK, S, P, O): (AggregatedEmployeeSubjectKind, EmployeeSubjectKind, EmploymentRelationshipPredicateKind, EmployerObjectKind)

(PK, S, P, O): (AggregatedEmploymentRelationshipPredicateKind, EmployeeSubjectKind, EmploymentRelationshipPredicateKind, EmployerObjectKind)

(OK, S, P, O): (AggregatedEmployerObjectKind, EmployeeSubjectKind, EmploymentRelationshipPredicateKind, EmployerObjectKind)

Rule:

Kinds “static schema / class” property definitions. Knowledge Layer Actor CSPO input.

(CK, SK, PK, OK): (AggregatedEmploymentContextKind, AggregatedEmployeeSubjectKind, AggregatedEmploymentRelationshipPredicateKind, AggregatedEmployerObjectKind)

Knowledge Layer: Activation Agent

Activation Agent (Interaction / Context Statements, DCI Layer). Transforms / Rules

DCI Layer: Interaction / Context Statements (Transforms, Rules)

Activation phase consumes Alignment Rule Statements, produces Context Statements.

Folding: From input Statements through Context Statements by means of Kinds Aggregation.

Unfolding: From aggregated Context Kind Statements into their originating Kind Statements.

Layer Types: Actor / Role / Context

Actor Statements:

Consumes Information Layer Property CSPO Quad Statements.

(C, S, P, O)

Consumes Information Layer Rule Statements as input:

(CK, SK, PK, OK): (AggregatedEmploymentContextKind, AggregatedEmployeeSubjectKind, AggregatedEmploymentRelationshipPredicateKind, AggregatedEmployerObjectKind) is an example of the source (C, S, P, O) of this layer.

Role Statements:

Layer Kinds.

Each layer calculates its corresponding Kinds from their input CSPO Statements from their above input layer aggregating inputs into their corresponding layer CSPO Kinds (Actor CSPO Statements in this layer).

(CK, S, P, O): (EmploymentRoleKind, AggregatedEmployeeSubjectKind, AggregatedEmploymentRelationshipPredicateKind, AggregatedEmployerObjectKind)

(SK, S, P, O): (EmployeeRoleKind, AggregatedEmployeeSubjectKind, AggregatedEmploymentRelationshipPredicateKind, AggregatedEmployerObjectKind)

(PK, S, P, O): (EmploymentRelationshipRoleKind, AggregatedEmployeeSubjectKind, AggregatedEmploymentRelationshipPredicateKind, AggregatedEmployerObjectKind)

(OK, S, P, O): (EmploymentProviderRoleKind, AggregatedEmployeeSubjectKind, AggregatedEmploymentRelationshipPredicateKind, AggregatedEmployerObjectKind)

Context Statements:

Kinds “static schema / class” property definitions.

(CK, SK, PK, OK) : (EmploymentRoleKind, EmployeeRoleKind,
EmploymentRelationshipRoleKind, EmploymentProviderRoleKind)

Streaming Agents

Augmentation Pipeline consumes streaming data of XML serialized layers Statements (XML Format).

Each layer consumes its corresponding layer encoded Statements from the messaging backbone. Then, augmentation is performed by means of merging input data with previously known data encoded in each layer domain format.

Each layer merged data is “unfolded” into the corresponding previous layer originating Statements. Then, merging is performed again with already known data in the format of the corresponding unfolded layer Statements. This is repeated until the source Data layer (CSPO) raw Statements layer is reached and merged.

The process continues in a fan in / fan out fashion, folding again merge results Statements into corresponding upper layer Statements formats, merging in each step again with the corresponding already known data of the layer and folding back again into another upper layer until Knowledge Context layer format (the topmost layer) is reached.

Each merge / unfold / merge; merge / fold / merge step consumes already known data of each layer for merging by means of a streaming endpoint. Each step also produces output Statements in the messaging backbone for further processing.

This approach functionally “applies” already known data to new / ingested data by means of merging in the sense of considering already known knowledge as templates, rules or an “upper” ontology for matching mechanisms for incorporating new knowledge into the knowledge base. This way, already known data behaves as “code” for new knowledge ingestion and augmentation.

Pipeline Layers Processing Statements:

Matching layers input Statements. Produced layers output Statements:

Data Layer (Aggregation):

Input: Raw (CSPO) Quad Statements.

Output: (CK; SK, PK, OK) Property Quad Statements.

Information Layer (Alignment):

Input: Axis Statements (Aggregation Data Layer outputs)

Output: Rule Statements.:

Knowledge Layer (Activation):

Input: Actor Statements (Alignment Information Layer outputs)

Output: Context Statements (DCI Pattern).

TODO: Determine Resource, Occurrence, Kind, Statement application semantics (functional)

TODO: Determine Statement traversal / destructuring for individual components contextual applications.

TODO: Determine Augmentation layers Statements into layers processing Statements.

TODO: Determine inferred layers processing statement types.

TODO: Determine primitives (raw triples SPO?)

Augmentation Pipeline

Each Augmentation Pipeline Service Agent consumes Statements from the messaging backbone, applies the processing defined for its layer, and publishes the resulting Statements back to the backbone using the Blackboard messaging pattern.

The pipeline is organized into three augmentation phases:

- **Aggregation (DOM Layer):** infers Context, Role, Type, and Property structures from Data / Instance Statements.
- **Alignment (Dimensional Layer):** performs ontology matching, links, dimensional and order inference, producing State, Transition, and Rule structures.
- **Activation (DCI Layer):** derives Contexts, Use Cases, and Interactions from Alignment Rules.

Each layer uses **Previously known Statements** representing the results of previous augmentation processing. These are the layer's inferred knowledge and become the basis for subsequent processing:

- Aggregation → **Property Statements**
- Alignment → **Rule Statements**
- Activation → **Context Statements**

Layer Statements are therefore not static configuration; they represent the current results of augmentation and can be consumed by other pipeline agents, including agents in the same phase.

Augmentation Pipeline Streams Processing

Each pipeline agent processes incoming Statement streams by matching them against the **Previously known Statements** inferred from previously aggregated knowledge. This Statements act as functional templates or rules: when an input Statement matches, the agent applies the corresponding processing Statement to produce new Statements.

The Processing Statement type is determined by the pipeline phase:

- **Aggregation:** Property Statements

- **Alignment:** Rule Statements
- **Activation:** Context Statements

Streaming augmentation combines the incoming **Input Statements** with the corresponding **Previously known Statements** at the same hierarchy level. Processing then occurs in two complementary phases:

1. **Unfolding** — decompose the input and processing Statements into their source Statements at progressively higher levels of the Statement hierarchy. At each level, the corresponding Statements are merged with layers of types of the previously known augmentation state until the layer's initial Statement type is reached.
2. **Folding** — aggregate the merged source Statements back down through the hierarchy, combining each result with the corresponding previous augmentation Statements until the layer's final Statement type is reached.

This allows an input Statement to be matched against known processing knowledge while preserving the hierarchical relationship between the Statement and the lower-level Statements from which it was composed.

The resulting stream operations therefore follow a common pattern:

Input Statements → **Match Processed Statements** → **Unfold / Merge** → **Fold / Produce** → **Output Processed Statements**

The specific merge mechanism remains an implementation choice and may use RDF, TMRM, FCA, CPPE-based, or Sets Model approaches. Each layer performs augmentation based on their merge approaches and implementation.

Aggregation Service Agent

Context / Role / Type inference in the Aggregation Layer.

Alignment Service Agent

Ontology Matching / Links / Dimensional / Order and Attributes inference in the Alignment Layer.

Activation Service Agent

Contexts (Use Cases) and behaviors (Interactions) in the Activation Layer.

Helper Services

Registry Service

Naming Service

Index Service

Possible Merge Approaches

Identity Resolution driven. TODO: Find the way of unambiguously identifying concepts (Resources, Occurrences, Statements, SPOs, Kinds) equivalent (in a given role occurrence in a given context).

Merge: Reify as Resources Occurrences, Statements, SPOs, Kinds. Resolve ID of Resources, Contexts (ID, Type), Occurrences (IDs, Contexts, Roles).

Aggregated Processing / Production Statements acts as Templates / Upper ontology alignment schemas.

Layers context merged entities propagate upstream (unfolding) and downstream (folding).

Identity Encoding:

CPPE Embeddings

Index API.

FCA / Set Assertion Statements:

(Context, Concept, Role, Occurrence);

Concept, Context, Role, Occurrence recursively instance of Set Statements.

Naming Scheme:

Name assignment for inferred entities.

Registry Mapping of Merge encodings.

RDF Merge

This approach relies on graph theory. It essentially performs a graph union of multiple RDF datasets. A critical function of an RDF merge is the standardization and renaming of "blank nodes" (anonymous resources) across different graphs to avoid collisions, resulting in a single, unified knowledge graph of subject-predicate-object triples.

Merge Statements as RDF graph patterns, combining compatible subjects, predicates, and objects while preserving graph identity and provenance. This provides a straightforward representation for stream-level graph merging and interoperability with RDF-based tooling.

TMRM Merge

TMRM merging is driven strictly by subject identity. If two topics (entities) across different data streams share the same subject locator, subject identifier, or item identifier, they are deterministically merged into a single topic. The resulting merged topic accumulates all the names, occurrences, and associations of the original topics.

Perform merging at the TMRM metamodel level, using Resources, Occurrences, Statements, Subjects, Predicates, Objects, and Kinds as the semantic structures being matched and consolidated. This provides the underlying subject-centric merge and identity model for the pipeline.

FCA Merge

This approach merges data at the conceptual level by combining "formal contexts" (mappings of objects to their attributes). When two concept lattices are merged, the FCA algorithm recalculates the matrix to build a new, unified lattice. This is highly effective for discovering new structural hierarchies and implicit relationships that weren't visible in the isolated data streams.

Treat matching Statements as formal contexts and merge compatible contexts to derive shared Properties and higher-level relationships. This is particularly suited to the Aggregation phase, where input Statements are consolidated into Property Statements.

CPPE Based FCA / Model Primitives Merge

Operating at the most atomic level of a data architecture, this approach aligns the fundamental structural primitives (e.g., core entities, base properties, and semantic rules) of the models. By merging the structural primitives *first*, the pipeline creates a standardized baseline context, which is then fed into an FCA merge to build highly accurate, normalized concept lattices.

Combine FCA context merging with CPPE/model primitives to support higher-level semantic matching and inference. This approach can be used in the Alignment phase to derive ontology matches, links, dimensions, ordering, and related Rule Statements.

Sets Model Based Merge

Rooted in classic set theory, this methodology uses strict mathematical operations (unions, intersections, and relative differences) to combine data. It treats data streams as collections of elements, making it an incredibly fast, deterministic, and highly scalable approach for deduplication and exact-match aggregations.

Represent Statement components and their relationships as sets and apply set operations such as union, intersection, difference, and compatibility matching. This provides a simple compositional mechanism that can be used as a lower-level merge primitive within the other approaches.

Integration into Streams Processing

To apply these distinct methodologies within the stream processing architecture described previously, they must be orchestrated across the **Unfolding (decomposition)** and **Folding (aggregation)** phases. Because each merge type excels at a different level of semantic complexity, they can be layered sequentially.

1. The Unfolding Phase (Decomposition & Identity)

During unfolding, complex incoming data must be broken down into its foundational components and matched against known processing statements.

- **Sets Model Merge (The Filter):** As the high-velocity stream enters, a Sets Model merge can be applied first to perform rapid deduplication (intersections) and isolate novel data (differences).
- **TMRM Merge (The Anchor):** Once the data is broken into discrete sets, TMRM is used to determine exact subject identity. By breaking complex input statements down until subject identifiers are found, the pipeline ensures that incoming data is accurately anchored to the correct foundational entities in the Aggregation Layer.

2. The Folding Phase (Aggregation & Inference)

During folding, the pipeline rebuilds the decomposed data, merging it with processing statements to infer new, higher-level contexts.

- **Model Primitives & CPPE Merge (The Baseline):** As folding begins, primitive merging aligns the foundational rules and properties (Property Statements). This ensures that disparate data streams are speaking the same structural language before complex relationships are inferred.
- **RDF Merge (The Graph Builder):** As statements are aggregated into the Alignment Layer, an RDF merge combines the localized graphs. This establishes the dimensional order and creates new rules (Rule Statements) by linking entities based on their standardized properties.
- **FCA Merge (The Activator):** Finally, as the data reaches the Activation Layer, FCA merging is applied to the newly unified RDF graphs. By analyzing the attributes and objects, FCA computes the final conceptual lattice, inferring the overarching use cases, behaviors, and Context Statements required by the system.

Combining the Approaches for Stream Processing

These approaches can be combined rather than treated as mutually exclusive. A practical streams-processing flow would be:

1. Receive and match

An agent receives an Input Statement and matches it against the applicable Processing and Production Statements.

2. Unfold and normalize

The matched Statements are unfolded into their source-layer Statements. TMRM or RDF can provide the graph representation, while Sets Model operations can perform basic component matching and compatibility checks.

3. Merge at the appropriate layer

The merge mechanism is selected according to the layer's responsibility:

- **Aggregation:** RDF/TMRM representation + FCA context merge → Property Statements.
- **Alignment:** TMRM/RDF representation + CPPE/FCA matching → Rule Statements.
- **Activation:** TMRM/RDF representation + model/set composition → Context Statements.

4. Fold and produce

The merged source Statements are folded back through the hierarchy. At each level, the resulting Statements are merged with the corresponding Production Statements until the layer's output Statement type is reached.

5. Publish the result

The resulting Production Statements are published to the messaging backbone and become available as processing knowledge for subsequent stream events or downstream agents.

This gives the pipeline a common processing pattern:

Input Stream → Match → Unfold → Merge → Infer → Fold → Production Stream

The important distinction is that **RDF/TMRM provide the graph and semantic representation**, while **FCA, CPPE/model primitives, and Sets Model operations provide progressively higher-level mechanisms for matching, merging, and inference**. This is consistent with the document's existing separation between the messaging/graph infrastructure and the Aggregation, Alignment, and Activation semantics.

I would also add a short note that **the merge approach is an implementation concern, while the Unfolding/Folding stream model is the common processing model**. That keeps the architecture flexible without committing the specification prematurely to one merge technology.

Leverage TMRM

Underlying graph metamodel.

Use the ISO Topic Maps Reference Model (TMRM) as the underlying graph metamodel for the Base Model, providing a subject-centric and implementation-independent foundation for

representing **Resource**, **Occurrence**, **Statement**, **Subject**, **Predicate**, **Object**, and their associated **Kind** structures. The Base Model can be mapped onto TMRM subject proxies and properties, with Statements providing the reified graph relationships and Occurrences providing the contextual roles in which Resources participate.

TMRM can therefore provide the semantic graph substrate and generic graph operations required by the service, while the Base Model and Augmentation Pipeline define the application-specific semantics and processing layers. In particular, the metamodel should support APIs for:

- **Resource / subject management:** create, identify, retrieve, update and merge graph resources/subject proxies, including stable URI-based identity.
- **Statement management:** create, retrieve, remove and traverse reified **S-P-O** Statements and their contextual Occurrences and roles.
- **Graph traversal:** navigate subjects, predicates, objects, occurrences, contexts and their relationships, including recursive traversal of Kinds and Statements.
- **Context-preserving access:** retrieve the statements and properties associated with a subject while preserving the context in which each relationship or occurrence was asserted.
- **Property and kind access:** expose the properties of a resource and the Statements defining **SubjectKind**, **PredicateKind** and **ObjectKind**, supporting the classification structures used by the Aggregation phase.
- **Pattern/query access:** match graph structures and Statement patterns so that Augmentation Pipeline agents can discover applicable processing Statements and apply them to input Statement streams.
- **Identity and merging:** provide subject identity/merging facilities that allow information about the same underlying subject from different sources or streams to be consolidated without losing its provenance/context.
- **Schema/constraint support:** expose the metamodel structures needed by higher-level services to validate or constrain the graph, while leaving the specific application constraints to the service model.

The TMRM layer is thus primarily responsible for **representing and accessing the semantic graph**, rather than implementing the Aggregation, Alignment or Activation semantics themselves. Those services consume and produce the graph structures through the metamodel APIs, allowing the same underlying graph representation to support the DOM, Dimensional and DCI layers described by the Augmentation Pipeline.

This also provides a natural foundation for the existing FCA and CPPE components: FCA can operate over the subject/property structures exposed by the graph, while CPPE can use Resource/Statement identities and identifiers as inputs to its numerical inference mechanisms.

Dimensional Features

Statement Predicates are the Type or Instance of an Action / Verb

:name instanceOf :naming

The Power Set of every attribute contains the sets of attributes of every possible class / concept. Includes relation defines an ordering over the power set. Power Sets (CSPO / Reified Types Sets). All possible Subjects, Predicate, Objects, Kinds Grouped Types. Order Relationship (Includes). CSPO Sets Intersection: All possible Statements. Grammar (production from possible rules).

Order and hierarchies are defined by subset / superset relationships. An object having attributes who are a superset of another object attributes is “extending” or “after” this object. Dimensional (Contexts) Features

Kinds, type / state hierarchy / order inference and materialization into Statements.

Alignment. Order (axis arrangements). State transitions flow. Infer and encode types / instances state transition networks on a given context on a given event:

- PCN (Previous, Current, Next) in axis / scope Graph Traversal Node Types: Individuals, States (occurrences role types), Associations (events). Order Hierachies. Contexts:
 - (Single, Marriage, Married);
 - (Marriage, Divorce, Marriage);
 - (Single, Married, Divorced);
 - (Junior, Promotion, Semisenior);
 - (Junior, Semisenior, Senior);
 - (Unemployed, Employment, Employee);
 - (Unemployed, Employed, Unemployed);
 - (John, successor, Peter);

Leveraging Grammars as an LLM Interaction tool

Possible contexts / functional transitions (behavior) as formal grammars rules / productions. Possible Prompts in context. Contexts Roles behaviors Flows. Inference context productions from grammars. Statements as grammars rules / productions.

Leverage RDF

Complete.

Leverage Sets Model

Complete.

Leverage FCA

Formal Concept Analysis for classification, types and attributes inference.

Subjects as Contexts:

Objects: Statement Predicates

Attributes: Statement Objects

Predicates as Contexts:

Objects: Statement Subjects

Attributes: Statement Objects

Objects as Contexts:

Objects: Statement Subjects

Attributes: Statement Predicates

Formal Concept Analysis (FCA) can be used as an inference model over the Base Model to derive classifications, types, attributes and shared properties from the Resource/Occurrence/Statement structures. FCA should operate on the graph structures exposed by the TMRM layer, treating the reified Statements as the source of object–attribute incidence relations rather than introducing a separate representation.

The three FCA contexts defined above provide complementary inference views:

- **Subject context:** Subjects are FCA contexts, Statement Predicates are objects, and Statement Objects are attributes. This allows the system to infer concepts representing groups of Subjects sharing predicate/object characteristics.
- **Predicate context:** Predicates are contexts, Statement Subjects are objects, and Statement Objects are attributes. This can infer common domains, ranges and behavioral/property patterns for predicates.
- **Object context:** Objects are contexts, Statement Subjects are objects, and Statement Predicates are attributes. This can infer classifications of objects based on the predicates through which they occur.

The resulting **formal concepts and concept lattice relations** can be treated as inferred **Kind**, **Property**, **Type** and potentially **Rule** Statements in the Augmentation Pipeline. In particular, FCA-derived common attributes can support the Aggregation phase by discovering candidate Kinds and Properties; the resulting classifications can then be consumed by Alignment to identify states, transitions and rules. FCA therefore acts primarily as an inference mechanism for discovering structure from existing statements, while the pipeline determines how those inferred structures are subsequently applied.

At the service level, FCA could expose basic APIs/functions such as:

- **Context construction:** build or update FCA contexts from Resources, Statements and their roles.
- **Concept discovery:** compute formal concepts and their extents/intents for Subjects, Predicates or Objects.
- **Classification:** determine the inferred Kinds/Types to which a Resource belongs based on its observed properties.
- **Attribute inference:** identify common or implied attributes shared by a set of Resources.

- **Similarity / generalization:** find more general or more specific concepts and compare Resources through their concept-lattice position.
- **Instance lookup:** retrieve Resources belonging to a given inferred concept or satisfying a concept's intent.
- **Incremental update:** update concepts when new Statements are added or removed, allowing inference to follow the evolving semantic graph.
- **Inference provenance:** associate inferred concepts or classifications with the Statements and contexts from which they were derived.

These operations should return normal Base Model Statements/resources where possible, allowing FCA results to participate in the same functional streams and messaging infrastructure as other augmentation outputs. FCA consequently becomes a reusable inference service rather than a separate data model, providing classification and attribute-discovery capabilities to the Aggregation and subsequent Alignment/Activation agents.

FCA-based Relational Schema Inference:

The system can infer relational schemas (rules or "upper concepts") from the structure of the data itself using FCA.

- **FCA Contexts for Relational Analysis:** We use three types of FCA contexts to analyze relationships from different perspectives:
 1. **Predicate-as-Context:** (FCA Objects: Statement Subjects, FCA Attributes: Statement Objects, FCA Context: Statement Predicate). This context reveals which types of subjects relate to which types of objects for a given predicate.
 2. **Subject-as-Context:** (FCA Objects: Statement Predicates, FCA Attributes: Statement Objects, FCA Context: Statement Subjects). This reveals all the relationships and objects associated with a given subject, defining its role.
 3. **Object-as-Context:** (FCA Objects: Statement Subjects, FCA Attributes: Statements Predicates, FCA Context: Statement Objects). This reveals all the subjects and actions that affect a given object.
- **Algorithm: Inferring Relational Schema:**
 1. **Select Context:** For a given predicate P (e.g., :worksOn), the Alignment Service constructs the Predicate-as-Context.
 2. **Build Lattice:** It uses an FCA library (e.g., fcalib) to compute the concept lattice from this context.
 3. **Identify Formal Concepts:** Each node in the lattice is a formal concept (A, B), where A is a set of subjects (the "extent") and B is the set of objects they all share (the "intent").
 4. **Materialize Schema:** Each formal concept represents an inferred relational schema or "upper concept". The system creates a new RDF class for this concept. For a concept where the extent is {dev1, dev2} (both :Developers) and the intent is {projA, projB} (both :Projects), the system can materialize a schema: :DeveloperWorksOnProject a rdfs:Class, :RelationalSchema ;

:hasDomain :Developer ;
:hasRange :Project .

Leverage CPPE

Algebraic inference by means of numerical prime numbers IDs embeddings.

Algebraic Semantic Embeddings (CPPE):

Prime Numbers Identifiers:

Every Resource is assigned a unique Prime Number ID (sequence) associated with its URI (via Registry Helper Service API).

Prime ID Embeddings:

Each Resource is assigned an unique incremental Prime Number Identifier.

For a given Resource occurrence in a given Statement context its Prime ID Embedding is calculated as the product of this occurrence Resource Prime ID Embedding with the Prime ID Embeddings of the other two parts of the Statement occurrences.

For example: given an object occurrences its Prime ID Embedding is the product of its Prime ID (Embedding) by the Prime ID (Embedding) of the other occurrences components.

Augmentation Layers. Stream Pipelines:

Aggregation, Alignment, Activation steps. Leverage Prime ID Embeddings for reactive functional composition.

CPPE. FCA-based Embeddings: A Deterministic Approach:

Resource Embeddings are calculated by the product of the Resource's Prime ID by the product of the Resource's Occurrences Attributes and Values Prime IDs.

We will replace LLM-based embeddings with deterministic, structural embeddings derived from FCA contexts and prime number products. This provides explainable similarity based on shared roles and relationships.

- **Contextual Prime Product Embedding (CPPE):** For any Occurrence (i.e., a resource in a specific statement), we can calculate an embedding based on its relational context.
 1. **Define FCA Contexts:** For a given relation (predicate), we can form an FCA context. Example: For the predicate :worksFor:
 - **Objects (G):** The set of all subjects of :worksFor statements (e.g., {id:Alice,

id:Bob})).

- **Attributes (M):** The set of all objects of :worksFor statements (e.g., {id:Google, id:StartupX}).
- 2. **Calculate Prime Product:** The CPPE for id:Google within the :worksFor context is the product of the primeIDs of all employees who work there. $CPPE(\text{Google}, \text{worksFor}) = \text{primeID}(\text{Alice}) * \text{primeID}(\text{Bob}) * \dots$
- **Similarity Calculation & Inference:**
 - **Similarity:** The similarity between two entities in the same context is the Greatest Common Divisor (GCD) of their CPPEs. $GCD(CPPE(\text{Google}), CPPE(\text{StartupX}))$ reveals the primeID product of their shared employees, giving a measure of personnel overlap.
 - **Relational Inference:** We can infer complex relationships. Consider the goal of finding an "uncle".
 1. Calculate the CPPE for "Person A" in the :brotherOf context (the product of their siblings' primes).
 2. Calculate the CPPE for "Person B" in the :fatherOf context (the product of their children's primes).
 3. If $GCD(CPPE_brotherOf(A), CPPE_fatherOf(B)) > 1$, it means A is the brother of B's father. The system can then materialize a new triple: (A, :uncleOf, ChildOfB). This inference is stored and queryable.

Relational Context Vectors

The core of this approach is the Relational Context Vector (RCV). For any given statement (a reified triple), we compute a vector of three BigInteger values, (S, P, O). Each component is a CPPE calculated from one of the three FCA context perspectives, providing a holistic numerical signature of the statement's role in the graph.

- **RCV Definition:** $RCV(\text{statement}) = (S, P, O)$
 - **S (Subject Context Embedding):** The CPPE of the statement's subject from the Subject-as-Context perspective. This number encodes everything the subject does. $S = \text{calculateCPPE}(\text{statement.subject}, \text{SubjectAsContext})$
 - **P (Predicate Context Embedding):** The CPPE of the statement's predicate from the Predicate-as-Context perspective. This number encodes every subject-object pair the predicate connects. $P = \text{calculateCPPE}(\text{statement.predicate}, \text{PredicateAsContext})$
 - **O (Object Context Embedding):** The CPPE of the statement's object from the Object-as-Context perspective. This number encodes everything that happens to the object. $O = \text{calculateCPPE}(\text{statement.object}, \text{ObjectAsContext})$
- **Implementation:** A Java record $RCV(\text{BigInteger } s, \text{BigInteger } p, \text{BigInteger } o)$. The Index Service is responsible for calculating and caching the RCV for every reified statement in the graph.

Schema Archetypes:

This dual representation is key to performing inference.

- **Instance RCV:** The RCV calculated for a specific, concrete statement (e.g., stmt_123: (dev:Alice, :worksOn, proj:Orion)) is its unique numerical signature. It represents a single data point.
- **Schema RCV (Archetype):** The RCV for a relational schema (e.g., the :DeveloperWorksOnProject schema) is an "archetype" vector. It is calculated by finding the Least Common Multiple (LCM) of the corresponding components of all instance RCVs that belong to that schema.
 - **Algorithm: calculateSchemaRCV(schemaURI)**
 1. Find all instance statements s_i where s_i `rdf:type` schemaURI.
 2. For each instance s_i , retrieve its cached $RCV_i = (S_i, P_i, O_i)$.
 3. Calculate the schema RCV components:
 - $S_schema = LCM(S_1, S_2, \dots, S_n)$
 - $P_schema = LCM(P_1, P_2, \dots, P_n)$
 - $O_schema = LCM(O_1, O_2, \dots, O_n)$
 4. The result $(S_schema, P_schema, O_schema)$ is the numerical archetype for the schema. The LCM ensures that the schema's numerical signature is "divisible" by all of its instances.

Subsumption:

Subsumption / Instance Checking (rdf:type):

- **Concept:** An instance belongs to a schema if the instance's RCV "divides into" the schema's RCV.
- **Algorithm: isInstanceOf(instanceRCV, schemaRCV)**
 1. Perform a component-wise modulo operation.
 2. `boolean isS = schemaRCV.s.mod(instanceRCV.s).equals(BigInteger.ZERO);`
 3. `boolean isP = schemaRCV.p.mod(instanceRCV.p).equals(BigInteger.ZERO);`
 4. `boolean isO = schemaRCV.o.mod(instanceRCV.o).equals(BigInteger.ZERO);`
 5. Return `isS && isP && isO`.
- **Use Case:** This is a high-speed, purely numerical method for checking type constraints, which can be performed in memory without a complex graph query.

Property Chains

Define composed relations (e.g., knowsLanguage) and leverage reasoners for closure. This section details the specific numerical algorithm for the `(:Developer)-[:worksOn]->(:Project)` and `(:Project)-[:usesLanguage]->(:Language) ==>`

(:Developer)-[:knowsLanguage]->(:Language) inference.

- Step 1: Define the Composition Operator
compose(RCV1, RCV2)
 - **Inferred Subject (S_inferred):** $S_inferred = RCV1.s * RCV2.o$
 - **Inferred Object (O_inferred):** $O_inferred = RCV1.s * RCV2.o$
 - **Inferred Predicate (P_inferred):** $P_inferred = RCV1.p * RCV2.p$
- **Step 2: Calculate Schema Archetypes**
 - The Index Service calculates archetypal RCVs for source schemas:
 - $RCV_worksOn_schema = (S_wo, P_wo, O_wo)$
 - $RCV_usesLang_schema = (S_ul, P_ul, O_ul)$
 - It then calculates the archetypal RCV for the inferred schema (knowsLanguage):
 - $S_kl = S_wo * O_ul$
 - $P_kl = P_wo * P_ul$
 - $O_kl = S_wo * O_ul$
 - This resulting $RCV_knowsLang_schema = (S_kl, P_kl, O_kl)$ is stored.
- Step 3: The Inference Algorithm at Query Time
A user asks: "Does dev:Alice know lang:Java?"
 1. **Retrieve Instance RCVs:** Retrieve RCV1 for (dev:Alice, :worksOn, proj:Orion) and RCV2 for (proj:Orion, :usesLanguage, lang:Java).
 2. **Calculate Hypothetical Instance RCV:** $RCV_hypothetical = compose(RCV1, RCV2)$.
 3. **Retrieve Schema Archetype:** Retrieve $RCV_knowsLang_schema$.
 4. **Perform Numerical Check:** boolean knows = $isInstanceOf(RCV_hypothetical, RCV_knowsLang_schema)$.
 5. **Result:** If knows is true, the inference is validated.

Querying and Traversal by Numerical Properties:

- **Find by Relational Role:** "Find all entities that have acted as a :Developer".
 - The query becomes: "Find all statements whose instanceRCV.s component divides $RCV_dev_schema.s$."
- **Traversal by Numerical Similarity:**
 - Start at stmt_A with RCV_A.
 - The next step: "Find stmt_B whose RCV_B has the highest GCD with RCV_A."
 - Allows traversal based on numerically similar relational contexts.

Graph Pipeline Specification: CPPE Algebra for Pipeline Stream Actions:

The **Contextual Prime Product Embedding (CPPE)** algebra treats every node in the graph not as a string or a pointer, but as a unique prime number. By leveraging the **Fundamental Theorem of Arithmetic**, relationships and sets are represented as products. This allows the Reactive Stream Pipeline to perform complex graph "joins" and "path traversals" using

simple CPU-native arithmetic.

1. Algebraic Definitions

Let $\mathcal{P}$ be the set of unique Prime IDs assigned to Resource URIs.

For any ContextPoint x :

- $\text{id}(x) \in \mathcal{P}$ (The singleton identity).
- $\text{emb}(x)$ (The cumulative embedding).

The Prime Product Rule:

The embedding of a triadic occurrence (c, o, a) —where c is Context, o is Object, and a is Attribute—is defined as the product of their respective recursive embeddings:

$$\text{emb}(c, o, a) = \text{emb}(c) \times \text{emb}(o) \times \text{emb}(a)$$

Because every factor is a prime (or a product of primes), the result is a unique "coordinate" in the integer space that encodes the entire lineage of the relationship.

2. Pipeline Stream Operators (The CPPE Monad)

In a reactive stream (e.g., `Flux<FCAContextPoint>`), the following algebraic operators are used for "Stream Action":

A. The "Contains" Operator (Sub-graph Filtering):

To check if a stream of Objects belongs to a certain Context C or possesses an Attribute A :

- **Logic:** $x \in \text{Context}(C) \iff \text{emb}(x) \pmod{\text{id}(C)} \equiv 0$
- **Reactive Implementation:**

```
// Filter objects that are part of the 'Accounting' context
objectStream.filter(obj ->
obj.getPrimeIDEmbedding().remainder(accountingID).equals(BigInteger.ZERO));
```

B. The "Intersection" Operator (Aggregation):

To find commonalities between two streams (finding the "Type" or "Concept"):

- **Logic:** $\text{Type}(x, y) = \text{gcd}(\text{emb}(x), \text{emb}(y))$
- **Stream Action:** The Greatest Common Divisor (GCD) of the embeddings of two nodes represents their shared FCA attributes/contexts.
- **Aggregation Use Case:** When the stream encounters multiple SPO triples, it calculates the "running GCD" of the attributes to infer the Schema (the most specific common super-type).

C. The "Axis Shift" Operator (Alignment):

Alignment predicts a value for a new context by calculating the "Ratio" of change between existing embeddings.

- **Logic:** $\text{Shift}(x, \Delta) = \text{emb}(x) \times \frac{\text{id}(\text{NewContext})}{\text{id}(\text{OldContext})}$
- **Stream Action:** By dividing by the "Old Axis" prime and multiplying by the "New Axis" prime, we re-project the object into a new coordinate.

3. Pipeline Stages Revisited via CPPE:

3.1 Aggregation (Algebraic Type Inference):

The Aggregation pipeline consumes raw SPO streams and produces **"Type Tensors"**.

1. **Input:** Stream of rotated triples (S, P, O) .
2. **Action:** For all x sharing attribute a , calculate the product $\prod \text{id}(x)$.
3. **Inference:** If $\text{emb}(x)$ is divisible by the product of a set of attribute primes, x is classified into that Type.
4. **Complexity:** $O(1)$ lookup via modulo instead of $O(N)$ graph search.

3.2 Alignment (Vector/Prime Space Prediction):

Alignment uses the **Prime Ratio** to detect structural similarity.

1. **Input:** Aggregated Types.
2. **Action:** Map the distance between objects O_1 and O_2 as $d = \frac{\text{emb}(O_1)}{\text{emb}(O_2)}$.
3. **Discovery:** If O_3 and O_4 share the same ratio d , the system aligns them as being "analogous" in different contexts (e.g., "CEO is to Company A what Principal is to School B").

3.3 Activation (State Transition Product):

Activation is the "Activation Energy" required to move from one prime state to another.

- **Transition Formula:** $\text{Transition} = \text{emb}(\text{Prev}) \otimes \text{emb}(\text{Next})$
- **Monadic Wrap:** The Occurrence Monad captures the prime state of an actor. When the stream processes a "Role" event, it multiplies the actor's prime ID by the role's prime ID.
- **Trigger:** If the resulting product matches a known "Success Pattern" (a pre-calculated product of requirements), the **Activation** is fired (e.g., triggering a webhook or a UI change).

Leverage W3C DIDs (Distributed Resource Identifiers)

Use W3C Decentralized Identifiers (DIDs) as a **distributed semantic registry mechanism** for Resources, Statements, Kinds, Schemas, Contexts and other semantic artifacts participating in the service. A DID provides a stable, globally addressable identifier whose associated DID Document can describe the identity, endpoints, metadata and verification mechanisms required to resolve and interact with the corresponding semantic resource.

Within the proposed architecture, the DID layer acts primarily as a **distributed identity and registry service**, while the TMRM/Base Model remains responsible for representing the semantic graph and its Resource/Occurrence/Statement structures. DIDs can therefore provide a common identity mechanism across independently managed data sources, semantic services and augmentation agents.

The Registry Service could expose APIs/functions such as:

- **Resource registration:** register a semantic Resource and associate it with a DID and its Base Model URI/identifier.
- **DID resolution:** resolve a DID to its current metadata, semantic endpoints and associated resource information.
- **Resource discovery:** discover semantic resources, schemas, Kinds, contexts or services by DID, type, capability or semantic relationship.
- **Identity mapping:** maintain mappings between external identifiers, source-specific URIs and canonical DIDs.
- **Document/version management:** publish and retrieve changes to the metadata describing a registered resource, supporting versioned semantic definitions.
- **Verification:** verify the provenance, controller and integrity of registered semantic resources or statements where appropriate.
- **Service endpoint discovery:** expose the APIs, messaging endpoints or graph services through which a registered semantic resource can be accessed.
- **Delegation/authorization:** provide a foundation for determining which agents or services are allowed to publish, modify or operate on registered semantic resources.
- **Provenance and lineage:** associate resources and semantic statements with their originating registries, sources, agents or augmentation processes.
- **Distributed synchronization:** propagate registry changes through the existing Event Bus so that local indexes and semantic services can react to resource registration, update or removal events.

A DID can consequently serve as the **distributed identity coordinate** for a semantic resource, while the Registry Service maintains the relationship between that identity and the actual semantic graph structures, endpoints and metadata. The Index Service can use these identifiers when maintaining searchable and numerical indexes, including the Prime ID mappings used by CPPE.

This separation allows the architecture to distinguish between **identity, registration and semantic representation**: DIDs identify and locate resources across administrative boundaries; the Registry Service manages their distributed registration and discovery; and the TMRM/Base Model represents their semantic relationships. Augmentation Pipeline agents can then consume resources discovered through the registry and produce Property, Rule and Context Statements as part of the existing Aggregation, Alignment and Activation pipeline.

The resulting API surface can therefore provide a common infrastructure for:

Register → Resolve → Discover → Verify → Synchronize → Consume

where the registered artifacts are not limited to data entities, but can include semantic schemas, Kinds, processing Statements, Contexts, service capabilities and other resources required to construct the distributed semantic application.

Example: Music catalog as a registered semantic resource

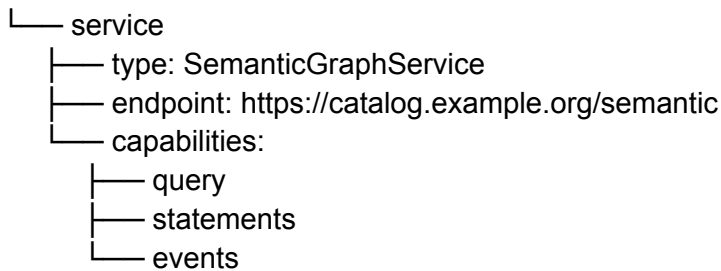
Suppose the semantic resource representing an external **Music Catalog** is registered under a DID. Its DID Document could expose a semantic graph endpoint such as:

<https://catalog.example.org/semantic>

The endpoint discovery mechanism tells the system: “**this DID represents a Music Catalog, and this is the service through which its semantic resources can be queried or consumed.**”

For example:

DID: did:example:music-catalog-123

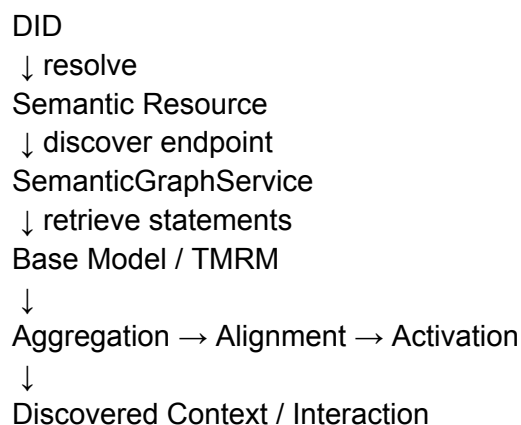


A local **Registry Service** resolves the DID and discovers the endpoint. The system can then consume statements such as:

(Album42, type, Album)
(Album42, digitalVariantOf, Product42)
(Artist7, created, Album42)
(Product42, availableIn, StoreA)

These statements enter the existing **Augmentation Pipeline**. The Aggregation Agent can infer the relevant types and properties; the Alignment Agent can recognize that **Album** can participate as a **Product**; and the Activation Agent can potentially discover contexts such as **Purchase** or **DigitalDownload**. This follows the document's existing model in which external data is converted to RDF/SPO statements and then processed through Aggregation, Alignment and Activation.

The important point is that **endpoint discovery decouples semantic identity from physical location**:



So, for example, the music store does not need to know beforehand that **Album42** is hosted by a particular catalog API. It can discover the resource through its DID, obtain the semantic service endpoint, ingest its statements, and let the rest of the system determine how that resource participates in the application's existing semantic model.

This also fits the document's proposed **Event Bus synchronization**: if the catalog publishes a new semantic statement—e.g. **Product42 availableIn StoreB**—the event can be consumed by the pipeline, causing the relevant properties, alignments or contexts to be recomputed or activated.

In that sense, **DID service endpoint discovery becomes the bridge between the distributed semantic ecosystem and the local semantic runtime**: it answers *“where/how can I access this semantic resource?”*, while TMRM, FCA, CPPE and the Augmentation Pipeline determine *“what does this resource mean, how does it relate to other resources, and what can the system do with it?”*

W3C Decentralized Identifiers (DIDs) can be used as a **distributed semantic registry mechanism** for identifying Resources and discovering the services through which their semantic representations and capabilities can be accessed. The DID layer provides the distributed identity and service-discovery mechanism, while the Base Model/TMRM remains responsible for representing Resources, Occurrences and Statements and their semantic relationships.

A registered semantic Resource exposes one or more service endpoints through its DID Document. For example, a music catalog could register an Album Resource as:

```
{  
  "id": "did:example:album-42",  
  "service": [  
    {  
      "id": "did:example:album-42#semantic",  
      "type": "SemanticResourceService",  
      "serviceEndpoint": "https://music.example/api/semantic/album-42",  
      "capabilities": [  
        "read",  
        "query",  
        "statements",  
        "subscribe"  
      ]  
    }  
  ]  
}
```

```
]
}
]
}
```

The DID therefore provides a stable distributed identity, while the service entry provides the means to discover and access the semantic resource. The exact DID Document structure and transport protocol are implementation choices to be defined by the service; the existing architecture already provides the corresponding Registry, Naming and Index helper-service roles.

A minimal Semantic Resource API could provide operations such as:

GET /resource/{did}

GET /resource/{did}/statements

POST /query

POST /statements

POST /subscribe

These APIs should expose the resource using the Base Model's semantic structures rather than requiring consumers to understand the resource's native backend representation. A Datasource Adapter can translate external APIs, databases or other formats into RDF/SPO Statements, which are then incorporated into the TMRM/Base Model. This is consistent with the existing requirement to transform arbitrary datasource formats into RDF SPO triples and with the TMRM APIs for resource management, statement access, traversal and pattern/query matching.

Example: Customer Buys Album

Consider a distributed music-store application. The following Resources may be registered independently:

did:example:customer-123

did:example:album-42

did:example:inventory-7

The Registry Service resolves their DIDs and discovers their semantic service endpoints. The services expose statements such as:

(Customer123, type, Customer)

(Customer123, state, Available)

(Album42, type, Product)

(Album42, state, InStock)

(Album42, availableIn, Inventory7)

These statements are ingested through the Datasource/semantic adapters and published through the existing messaging backbone. The TMRM/Base Model provides their common semantic representation and context-preserving access.

The Augmentation Pipeline can then process the distributed information:

Distributed DID Resources



Service Endpoint Discovery



Semantic Resource APIs



RDF/SPO / Context Statements



TMRM / Base Model



Aggregation

Customer / Product / Properties



Alignment

Available + InStock → Purchase Rule



Activation

Purchase Context

↓

Purchase Interaction

The Activation Agent can materialize an interaction such as:

Purchase#123

(Customer123, purchases, Album42)

The resulting interaction can then be reduced to deterministic semantic operations, for example:

CreateLink(Customer123, owns, Album42)

RemoveLink(Album42, availableIn, Inventory7)

EmitEvent(AlbumSold)

This follows the document's model in which Activation produces Context Statements representing use cases and behaviors, while higher-level behaviors can ultimately be reduced to primitive graph transformations and events.

The DID mechanism therefore does not execute the **Purchase** operation itself. Its role is to provide the **distributed identity, discovery and access point** from which semantic resources can be consumed. The Semantic Resource API provides access to their statements and capabilities; TMRM provides the semantic graph substrate; Aggregation, Alignment and Activation interpret those statements; and the semantic execution layer performs the resulting graph transformations.

At the service level, the resulting architecture can be summarized as:

DID

↓

Registry / DID Resolution

↓

Service Endpoint Discovery

↓

Semantic Resource API

↓

Datasource Adapter

↓

TMRM / Base Model



Aggregation → Alignment → Activation



Context / Interaction



Semantic Execution / Events

DIDs consequently provide a mechanism for turning the proposed system from a locally hosted semantic graph into a **distributed semantic application environment**, in which independently managed Resources can publish discoverable semantic services and participate in the same Resource, Statement, Augmentation and Activation infrastructure.

In the architecture described in the document, **the owner/operator of the distributed semantic resource implements its Semantic Resource API**. The DID mechanism does not implement the business functionality; it provides the identity and discovery mechanism that allows the rest of the system to find that API.

For the **Customer buys Album** example:

- **Music Catalog / Album owner** implements the API for **did:example:album-42**, exposing album statements and capabilities.
- **Customer service** implements the API for **did:example:customer-123**, exposing the customer's relevant semantic state.
- **Inventory service** implements the API for **did:example:inventory-7**, exposing inventory statements and potentially inventory operations.
- The **Registry Service** registers these DIDs and makes their service endpoints discoverable.
- The **Datasource/semantic adapters** translate the underlying systems' data into the document's Resource/Statement model.
- The **semantic runtime**—TMRM, Aggregation, Alignment and Activation agents—consumes those APIs; it does not need to know how the underlying resource is implemented. The document explicitly separates datasource integration, registry services, and the augmentation pipeline.

So architecturally:

Resource Owner



| implements



Semantic Resource API

|

| registered as



DID + DID Document

|

| discovered by



Registry / Semantic Runtime

|



TMRM → Aggregation → Alignment → Activation

There is one important distinction for the **Purchase operation**. The Album resource does not necessarily implement `purchase()` itself. The document's model suggests that **behavior belongs to semantic Contexts and Interactions**, rather than being owned by individual data objects.

Thus, the Album service might expose **data capabilities** such as:

`getStatements()`

`query()`

`subscribe()`

while a **Purchase/Store service** could expose the executable capability:

`purchase(customerDID, albumDID)`

The Activation Agent discovers that this capability is applicable, and the Purchase service executes the resulting semantic primitives.

In short: the resource owner implements the resource API; the Registry/DID infrastructure publishes and discovers it; and the semantic runtime orchestrates those distributed capabilities. The uploaded document does not yet specify which concrete component should host the executable `Purchase` API, so that is an architectural decision still to be defined.

Yes. This changes the architecture in an important way: **the DID identifies a domain service, rather than exposing an API for manipulating an individual resource.**

For the document's model, I think this is actually a better fit. The DID becomes the distributed identity of a **business capability/service**, while the resources (**Customer**, **Album**, **Inventory**) remain semantic entities referenced by that service.

Example: Music Store service

Instead of:

did:example:album-42

└─ /semantic/album-42

└─ getStatements()

└─ query()

the domain could expose:

did:example:music-store-7

└─ MusicStoreService

└─ discoverProducts()

└─ purchaseAlbum()

└─ reserveInventory()

└─ createOrder()

└─ subscribeToSales()

The DID Document would therefore advertise **business capabilities**:

```
{  
  
  "id": "did:example:music-store-7",  
  
  "service": [  
  
    {  
  
      "id": "did:example:music-store-7#api",  
  
      "type": "MusicStoreDomainService",  
  
      "serviceEndpoint": "https://store.example/api",  
  
      "capabilities": [  
  
        "discoverProducts",  

```

```
"purchaseAlbum",  
"reserveInventory",  
"createOrder"  
]  
}  
]  
}
```

The important shift is:

**The DID identifies “who provides this business capability?” rather than
“where is this particular resource?”**

The API becomes domain-driven

The API would consequently look more like:

POST /products/query

POST /purchases

POST /inventory/reservations

POST /orders

GET /events

For example:

POST /purchases

```
{  
  "customer": "did:example:customer-123",  
  "product": "did:example:album-42"  
}
```

The service implementation then uses its own domain logic and semantic graph to determine whether the purchase is valid.

It might retrieve:

Customer123 → state → Available

Album42 → type → Product

Album42 → state → InStock

and invoke the semantic pipeline:

Purchase Request



Domain Service



TMRM / Semantic Graph



Aggregation



Alignment



Activation



Purchase Context



Execution

This is particularly consistent with the document's later distinction between **data**, **contexts**, and **interactions**. A **Purchase** is not merely a property of an Album; it is a domain interaction involving Customer, Product, Inventory and potentially Order.

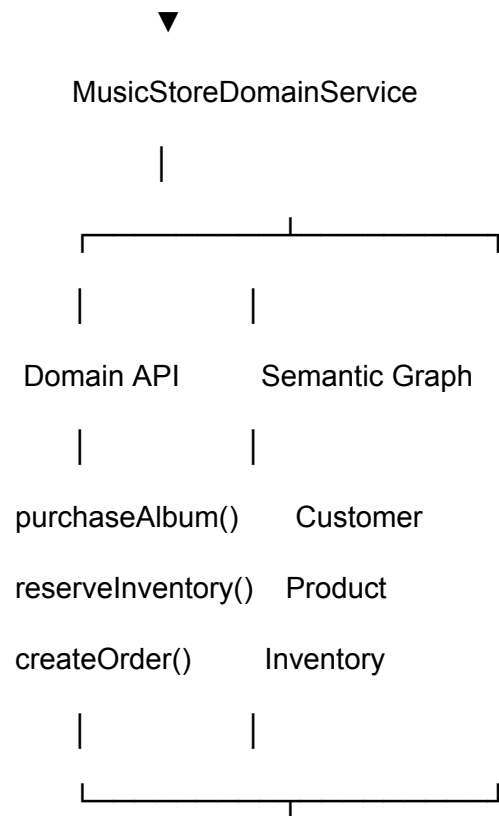
Where the semantic model fits

The domain API does **not replace** the semantic layer. Instead, the API becomes the **business-facing boundary over the semantic runtime**.

For example:

DID

|



▼

Aggregation / Alignment

▼

Activation

▼

Semantic Execution

Thus `purchaseAlbum()` is a **business capability**, while the semantic graph determines the entities, states, relationships and Context under which that capability can be activated.

This also gives DIDs a stronger role

A distributed application could discover a suitable service by DID:

did:example:music-store-7

↓

MusicStoreDomainService

↓

capability: purchaseAlbum

↓

endpoint: `https://store.example/api/purchases`

Another application could have its own implementation:

`did:example:another-store-9`

↓

`MusicStoreDomainService`

↓

capability: `purchaseAlbum`

↓

endpoint: `https://another.example/api/purchases`

Both can expose the **same semantic business capability**, while their internal implementations are completely different.

This is where the approach connects strongly to the document's idea of **semantic composition**: the runtime can discover that independently defined domains expose compatible concepts and interactions, align them, and potentially compose them into a larger application.

The resulting principle

I would therefore define the architecture as:

DIDs identify distributed domain services and advertise their semantic business capabilities. Domain APIs provide the invocation boundary for those capabilities. The semantic graph provides the shared representation of the domain entities and their state. Aggregation, Alignment and Activation determine how a requested business interaction maps onto that semantic graph, while deterministic semantic primitives perform the resulting state changes.

This is arguably a better match for the document than a purely resource-oriented DID API because the document ultimately aims at **contexts, interactions and executable behaviors**, not merely distributed CRUD over resources.

Appendix — Layered Statement Folding, Unfolding, and TreeGraph Representation

A. Purpose

This appendix elaborates the **Augmentation Layers Domain Model** through a concrete *Music Store* example. It describes how independently expressed **Music** and **Store** domain statements can be merged into a composite, executable semantic model through the layered **Data → Information → Knowledge** pipeline, and how the resulting higher-layer statements can subsequently be unfolded back to their composing lower-layer statements.

The appendix also proposes an XML/TreeGraph representation capable of preserving the complete folded/unfolded structure for downstream and upstream processing.

The approach follows the document's existing model in which lower-layer statements are aggregated into higher-layer statement types, while higher-layer statements can be decomposed back into their originating statements. The pipeline operates as a streaming merge/fold/unfold process over statements.

B. Music Store as a Composed Domain

The **Music** and **Store** domains may initially provide independent CSPO statements.

Example Music-domain input:

(Catalog, Album42, type, Album)

(Catalog, Album42, createdBy, Artist7)

(Catalog, Album42, containsTrack, Track421)

(Catalog, Track421, genre, Rock)

Example Store-domain input:

(StoreCatalog, Product42, type, Product)

(StoreCatalog, Product42, availableIn, StoreA)

(StoreCatalog, Product42, inventoryState, InStock)

(StoreCatalog, Customer123, type, Customer)

(StoreTransactions, Customer123, asksFor, Product42)

The two domains do not need to have been designed as a single ontology. Through aggregation, alignment and identity/semantic matching, compatible structures can be composed into a **Music Store** semantic model.

For example, an Album may be aligned with a Product, while Music-specific properties such as Artist and Track are retained alongside Store-specific properties such as Price, Inventory and Availability. This follows the document's Music Store example, where Music concepts

such as *Artist*, *Album*, *Track*, *Genre* are composed with Store concepts such as *Product*, *Inventory*, *Supplier*, *Order*, *Customer*.

The resulting model can consequently express a structure such as:

MusicProduct

└─ Album

└─ createdBy → Artist

└─ containsTrack → Track

└─ availableIn → Store

└─ inventoryState → InStock

The important distinction is that **Music Store is the result of semantic composition**, rather than necessarily being a predefined domain schema.

C. Data Layer — Instance, Type and Property

The Data layer performs **Aggregation**.

Its statement types are:

Instance

Type

Property

The layer consumes raw Data/Instance statements and aggregates them into progressively higher-level Kind and Property structures.

C.1 Instance statements

The source CSPO statements are concrete observations:

(Album42, type, Album)

(Album42, createdBy, Artist7)

(Album42, containsTrack, Track421)

(Product42, type, Product)

(Product42, availableIn, StoreA)

(Product42, inventoryState, InStock)

These are merged with other known statements and used to infer recurring structures.

C.2 Type statements

The resulting structures can establish Kinds such as:

AlbumKind

ProductKind

MusicProductKind

ArtistKind

TrackKind

StoreKind

CustomerKind

For example:

MusicProductKind

type → Album

createdBy → Artist

containsTrack → Track

availableIn → Store

inventoryState → InventoryState

C.3 Property statements

The recurring Kind relationships are folded into Property statements describing the static structure of the resulting domain:

MusicProductProperty

hasArtist

hasTrack

hasPrice

availableIn

inventoryState

Thus the Data layer answers:

What entities, kinds and structural properties emerge from merging the Music and Store data?

D. Information Layer — Axis, State and Rule

The Information layer performs **Alignment**.

Its statement types are:

Axis

State

Rule

The document describes this stage as transforming Data-layer Property structures into dimensional arrangements, states and rules through matching, completion and inference.

D.1 Axis

A composite Music Store axis can establish dimensions such as:

MusicProduct × Store × Availability × Customer

D.2 State

Concrete observations can then be represented as states:

Album42

inventoryState → InStock

Album42

availableIn → StoreA

Customer123

asksFor → Album42

These may be folded into a composite state:

AvailableMusicProductState

representing:

MusicProduct

+ Album

+ Product

+ InStock

+ AvailableIn(Store)

D.3 Rule

Compatible states can subsequently form a Rule:

MusicStorePurchaseRule

IF

Customer requests Product

AND Product is InStock

AND Product is availableIn Store

THEN

Purchase is possible

The Rule is therefore an upper-layer semantic structure derived from the lower-layer properties and states rather than necessarily being an independently programmed business rule.

The Information layer answers:

How do the structures from Music and Store align, what states can they form, and what rules follow from those states?

E. Knowledge Layer — Actor, Role and Context

The Knowledge layer performs **Activation**.

Its statement types are:

Actor

Role

Context

Information-layer Rules are folded into Context structures capable of representing executable interactions.

For the Music Store example:

Actors:

Customer123

Album42 / Product42

StoreA

Inventory7

Order77

can participate in roles such as:

CustomerRole

ProductRole

StoreRole

InventoryRole

OrderRole

These can be composed into:

MusicStorePurchaseContext

which can expose an executable interaction such as:

Purchase

└─ CreateOrder

└─ ReserveInventory

└─ RecordPurchase

└─ EmitEvent

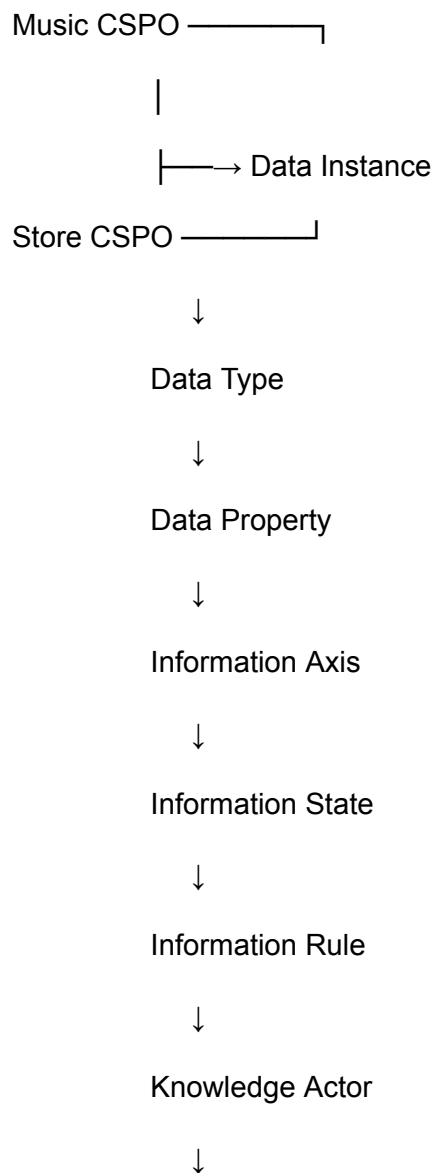
This corresponds to the document's broader example in which a semantic structure involving Customer, Product and Inventory produces a potential **Sale** Context and executable interactions such as **CreateOrder**, **ReserveInventory** and **GenerateInvoice**.

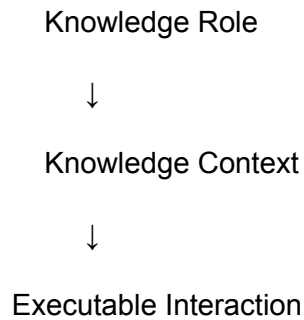
The Knowledge layer therefore answers:

Given the aligned Music Store semantics, what executable context and interaction can be activated?

F. Complete Folding Chain

The resulting composition can be summarized as:





Thus the **Music Store executable semantic model** is progressively folded from lower-level observations.

This follows the document's principle that each layer consumes the previous layer's statement structure and produces the corresponding higher-level statement structure.

G. Unfolding / De-aggregation

The same structure supports the reverse operation.

Starting with:

MusicStorePurchaseContext

the runtime can unfold it into:

CustomerRole

ProductRole

StoreRole

PurchaseRole

which can be unfolded into the Information structures:

MusicStorePurchaseRule

AvailableMusicProductState

which can be unfolded into Data structures:

MusicProductProperty

MusicProductKind

StoreKind

and ultimately into the source CSPO statements:

(Album42, type, Album)

(Album42, createdBy, Artist7)

(Album42, containsTrack, Track421)

(Product42, type, Product)

(Product42, availableIn, StoreA)

(Product42, inventoryState, InStock)

(Customer123, asksFor, Product42)

The important property is that unfolding does not merely discard the higher-level statement. It exposes the **statements that composed it**, allowing each recovered statement to be merged with the already-known statements at its own layer before continuing upstream.

The document explicitly describes this as a backwards process in which deeper statements are merged and unfolded until the raw S/P/O statements are reached.

H. TreeGraph XML Representation

The document proposes **TreeGraph** as the pipeline message format, with nested layer statements representing composition.

A suitable representation is therefore a nested **statement** structure:

```
<statement
```

```
  id="K-CONTEXT-001"
```

```
  layer="knowledge"
```

```
  type="context"
```

```
  kind="MusicStorePurchaseContext"
```

```
  operation="fold">
```

```
</c>
```

```
  <kindRef ref="MusicStorePurchaseContextKind"/>
```

</c>

<s>

<resource ref="Customer123"/>

</s>

<p>

<predicate ref="purchases"/>

</p>

<o>

<resource ref="Album42"/>

</o>

<composedOf>

<statement

id="I-RULE-001"

layer="information"

type="rule"

kind="MusicStorePurchaseRule">

<c>

<kindRef ref="MusicStorePurchaseRuleKind"/>

</c>

<s>

<kindRef ref="CustomerKind"/>

</s>

<p>

<kindRef ref="PurchasePredicateKind"/>

</p>

<o>

<kindRef ref="MusicProductKind"/>

</o>

<composedOf>

<statement

id="I-STATE-001"

layer="information"

type="state"

kind="AvailableMusicProductState">

<s>

<resource ref="Album42"/>

</s>

<p>

<predicate ref="inventoryState"/>

</p>

<o>

<value>InStock</value>

</o>

<composedOf>

<statement

id="D-PROPERTY-001"

layer="data"

type="property"

kind="MusicProductProperty">

<composedOf>

<statement

id="D-INSTANCE-001"

layer="data"

type="instance"

source="music">

<c ref="MusicCatalog"/>

<s ref="Album42"/>

<p ref="type"/>

<o ref="Album"/>

</statement>

<statement

id="D-INSTANCE-002"

layer="data"

type="instance"

source="store">

<c ref="StoreCatalog"/>

<s ref="Album42"/>

<p ref="inventoryState"/>

<o value="InStock"/>

</statement>

</composedOf>

</statement>

</composedOf>

</statement>

</composedOf>

</statement>

</composedOf>

</statement>

This representation preserves the **semantic statement itself** and the **composition relationship through which it was produced**.

I. Statement Identity and Graph References

Although TreeGraph is naturally nested, the underlying semantic structure may be a graph rather than a strict tree because one lower-layer statement can contribute to multiple higher-layer statements.

For example:

(Album42, inventoryState, InStock)

could contribute simultaneously to:

AvailableMusicProductState

PurchasableProductState

MusicStorePurchaseRule

Consequently, a production representation should support references:

<composedOf>

 <statementRef ref="D-INSTANCE-002"/>

</composedOf>

rather than requiring every statement to be duplicated.

The underlying structure can therefore be understood as a **DAG serialized as a TreeGraph**, with stable statement IDs preserving semantic identity.

This is consistent with the document's emphasis on statement identity, recursive traversal, merging and context-preserving graph access.

J. Fold/Unfold Processing Metadata

The TreeGraph can additionally preserve how a statement was produced:

<fold

 id="FOLD-001"

agent="AggregationAgent"

operation="aggregate">

<input ref="D-INSTANCE-001"/>

<input ref="D-INSTANCE-002"/>

<output ref="D-PROPERTY-001"/>

</fold>

followed by:

<fold

id="FOLD-002"

agent="AlignmentAgent"

operation="align">

<input ref="D-PROPERTY-001"/>

<output ref="I-RULE-001"/>

</fold>

and:

<fold

id="FOLD-003"

agent="ActivationAgent"

operation="activate">

<input ref="I-RULE-001"/>

<output ref="K-CONTEXT-001"/>

</fold>

This allows the representation to retain not only **what knowledge exists**, but also **which lower-layer statements contributed to it and which transformation produced it**.

It is particularly appropriate to the document's blackboard/stream-processing model, where agents merge incoming statements with already-known statements and publish the resulting augmented statements for further processing.

K. Layer Transformation Summary

The resulting transformation model is:

Layer	Statement types	Main operation	Output
Data	Instance → Type → Property	Aggregation / Folding	Structural semantic model
Information	Axis → State → Rule	Alignment / Folding	Dimensional/state/rule model
Knowledge	Actor → Role → Context	Activation / Folding	Executable semantic context
Reverse	Context → ... → Instance	Unfolding / De-aggregation	Source/composing statements

The forward path is:

- CSPO
- Type
- Property
- Axis
- State
- Rule
- Actor

→ Role

→ Context

The reverse path is:

Context

→ Role

→ Actor / Rule

→ State / Axis

→ Property

→ Type

→ CSPO

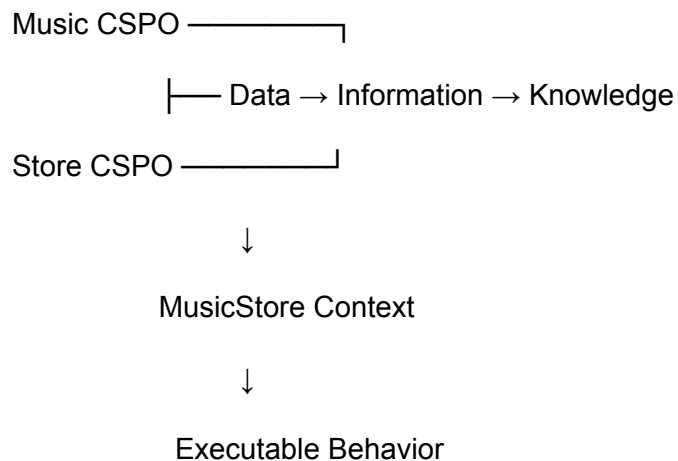
Each intermediate statement can therefore be treated simultaneously as:

1. **knowledge at its own layer,**
 2. **input to the next layer,**
 3. **a composition of lower-layer knowledge, and**
 4. **a decomposition target for upstream processing.**
-

L. Architectural Principle

The combined model can consequently be summarized as:

FOLD / AGGREGATE



UNFOLD / DE-AGGREGATE

MusicStore Context



Information / Data



Source CSPO

The central principle is that **the executable Music Store model does not have to be a separately authored artifact**. It can emerge from the composition of independently sourced semantic statements, with each augmentation layer adding a new level of abstraction while retaining the lower-level statements that justify that abstraction.

The TreeGraph representation makes this layering explicit and makes the same message structure usable for both directions:

fold upward to infer and activate; unfold downward to explain, trace, transform, or recover source statements.

This is consistent with the document's existing description of lower-layer composition, upper-layer aggregation, upstream decomposition, and the use of TreeGraph/XSLT for nested statement processing.

Note: the exact XML vocabulary, identity-resolution rules, and reference/graph semantics proposed in this appendix are design elaborations of the document's TreeGraph and folding/unfolding concepts; they are not specified as a finalized XML schema in the source document.

Appendix — Layered Statement Folding, Unfolding, and TreeGraph Representation

A. Purpose

This appendix elaborates the **Augmentation Layers Domain Model** through a concrete *Music Store* example. It describes how independently expressed **Music** and **Store** domain statements can be merged into a composite, executable semantic model through the layered **Data → Information → Knowledge** pipeline, and how the resulting higher-layer statements can subsequently be unfolded back to their composing lower-layer statements.

The appendix also proposes an XML/TreeGraph representation capable of preserving the complete folded/unfolded structure for downstream and upstream processing.

The approach follows the document's existing model in which lower-layer statements are aggregated into higher-layer statement types, while higher-layer statements can be decomposed back into their originating statements. The pipeline operates as a streaming merge/fold/unfold process over statements.

B. Music Store as a Composed Domain

The **Music** and **Store** domains may initially provide independent CSPO statements.

Example Music-domain input:

(Catalog, Album42, type, Album)

(Catalog, Album42, createdBy, Artist7)

(Catalog, Album42, containsTrack, Track421)

(Catalog, Track421, genre, Rock)

Example Store-domain input:

(StoreCatalog, Product42, type, Product)

(StoreCatalog, Product42, availableIn, StoreA)

(StoreCatalog, Product42, inventoryState, InStock)

(StoreCatalog, Customer123, type, Customer)

(StoreTransactions, Customer123, asksFor, Product42)

The two domains do not need to have been designed as a single ontology. Through aggregation, alignment and identity/semantic matching, compatible structures can be composed into a **Music Store** semantic model.

For example, an Album may be aligned with a Product, while Music-specific properties such as Artist and Track are retained alongside Store-specific properties such as Price, Inventory and Availability. This follows the document's Music Store example, where Music concepts such as *Artist*, *Album*, *Track*, *Genre* are composed with Store concepts such as *Product*, *Inventory*, *Supplier*, *Order*, *Customer*.

The resulting model can consequently express a structure such as:

MusicProduct

└─ Album

|— createdBy → Artist

|— containsTrack → Track

|— availableIn → Store

└— inventoryState → InStock

The important distinction is that **Music Store is the result of semantic composition**, rather than necessarily being a predefined domain schema.

C. Data Layer — Instance, Type and Property

The Data layer performs **Aggregation**.

Its statement types are:

Instance

Type

Property

The layer consumes raw Data/Instance statements and aggregates them into progressively higher-level Kind and Property structures.

C.1 Instance statements

The source CSPO statements are concrete observations:

(Album42, type, Album)

(Album42, createdBy, Artist7)

(Album42, containsTrack, Track421)

(Product42, type, Product)

(Product42, availableIn, StoreA)

(Product42, inventoryState, InStock)

These are merged with other known statements and used to infer recurring structures.

C.2 Type statements

The resulting structures can establish Kinds such as:

AlbumKind

ProductKind

MusicProductKind

ArtistKind

TrackKind

StoreKind

CustomerKind

For example:

MusicProductKind

type $\rightarrow$ Album

createdBy $\rightarrow$ Artist

containsTrack $\rightarrow$ Track

availableIn $\rightarrow$ Store

inventoryState $\rightarrow$ InventoryState

C.3 Property statements

The recurring Kind relationships are folded into Property statements describing the static structure of the resulting domain:

MusicProductProperty

hasArtist

hasTrack

hasPrice

availableIn

inventoryState

Thus the Data layer answers:

What entities, kinds and structural properties emerge from merging the Music and Store data?

D. Information Layer — Axis, State and Rule

The Information layer performs **Alignment**.

Its statement types are:

Axis

State

Rule

The document describes this stage as transforming Data-layer Property structures into dimensional arrangements, states and rules through matching, completion and inference.

D.1 Axis

A composite Music Store axis can establish dimensions such as:

MusicProduct × Store × Availability × Customer

D.2 State

Concrete observations can then be represented as states:

Album42

inventoryState → InStock

Album42

availableIn → StoreA

Customer123

asksFor → Album42

These may be folded into a composite state:

AvailableMusicProductState

representing:

MusicProduct

+ Album

+ Product

+ InStock

+ AvailableIn(Store)

D.3 Rule

Compatible states can subsequently form a Rule:

MusicStorePurchaseRule

IF

Customer requests Product

AND Product is InStock

AND Product is availableIn Store

THEN

Purchase is possible

The Rule is therefore an upper-layer semantic structure derived from the lower-layer properties and states rather than necessarily being an independently programmed business rule.

The Information layer answers:

How do the structures from Music and Store align, what states can they form, and what rules follow from those states?

E. Knowledge Layer — Actor, Role and Context

The Knowledge layer performs **Activation**.

Its statement types are:

Actor

Role

Context

Information-layer Rules are folded into Context structures capable of representing executable interactions.

For the Music Store example:

Actors:

Customer123

Album42 / Product42

StoreA

Inventory7

Order77

can participate in roles such as:

CustomerRole

ProductRole

StoreRole

InventoryRole

OrderRole

These can be composed into:

MusicStorePurchaseContext

which can expose an executable interaction such as:

Purchase

└─ CreateOrder

└─ ReserveInventory

└─ RecordPurchase

└─ EmitEvent

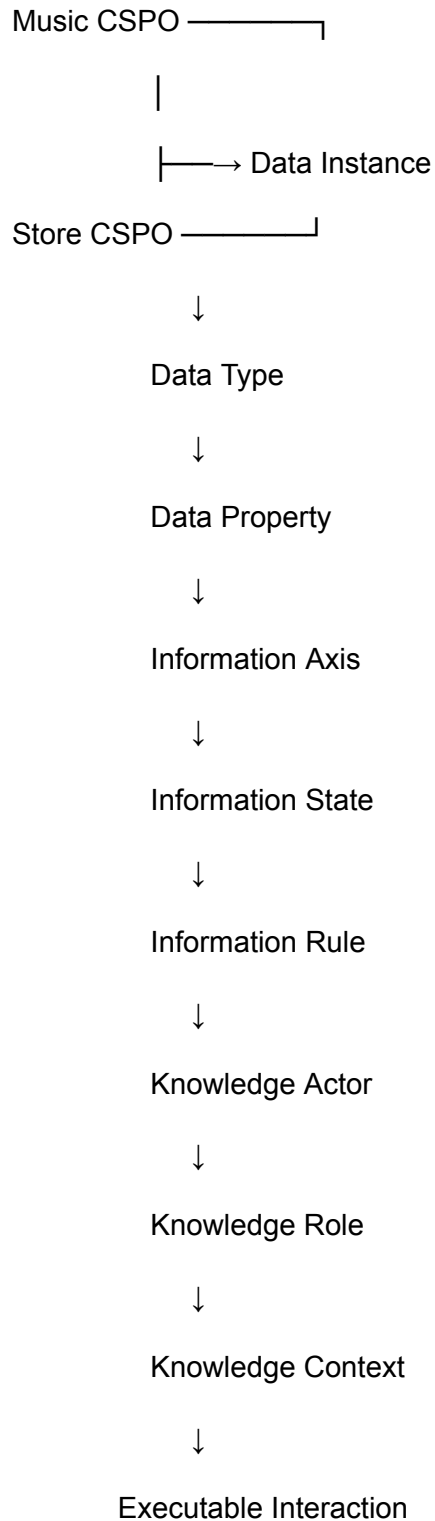
This corresponds to the document's broader example in which a semantic structure involving Customer, Product and Inventory produces a potential **Sale** Context and executable interactions such as *CreateOrder*, *ReserveInventory* and *GenerateInvoice*.

The Knowledge layer therefore answers:

Given the aligned Music Store semantics, what executable context and interaction can be activated?

F. Complete Folding Chain

The resulting composition can be summarized as:



Thus the **Music Store executable semantic model** is progressively folded from lower-level observations.

This follows the document's principle that each layer consumes the previous layer's statement structure and produces the corresponding higher-level statement structure.

G. Unfolding / De-aggregation

The same structure supports the reverse operation.

Starting with:

MusicStorePurchaseContext

the runtime can unfold it into:

CustomerRole

ProductRole

StoreRole

PurchaseRole

which can be unfolded into the Information structures:

MusicStorePurchaseRule

AvailableMusicProductState

which can be unfolded into Data structures:

MusicProductProperty

MusicProductKind

StoreKind

and ultimately into the source CSPO statements:

(Album42, type, Album)

(Album42, createdBy, Artist7)

(Album42, containsTrack, Track421)

(Product42, type, Product)

(Product42, availableIn, StoreA)

(Product42, inventoryState, InStock)

(Customer123, asksFor, Product42)

The important property is that unfolding does not merely discard the higher-level statement. It exposes the **statements that composed it**, allowing each recovered statement to be merged with the already-known statements at its own layer before continuing upstream.

The document explicitly describes this as a backwards process in which deeper statements are merged and unfolded until the raw S/P/O statements are reached.

H. TreeGraph XML Representation

The document proposes **TreeGraph** as the pipeline message format, with nested layer statements representing composition.

A suitable representation is therefore a nested **statement** structure:

```
<statement
  id="K-CONTEXT-001"
  layer="knowledge"
  type="context"
  kind="MusicStorePurchaseContext"
  operation="fold">

  <c>

    <kindRef ref="MusicStorePurchaseContextKind"/>

  </c>

  <s>

    <resource ref="Customer123"/>

  </s>

  <p>
```

<predicate ref="purchases"/>
</p>

<o>
 <resource ref="Album42"/>
</o>

<composedOf>

<statement
 id="I-RULE-001"
 layer="information"
 type="rule"
 kind="MusicStorePurchaseRule">

<c>
 <kindRef ref="MusicStorePurchaseRuleKind"/>
</c>

<s>
 <kindRef ref="CustomerKind"/>
</s>

<p>
 <kindRef ref="PurchasePredicateKind"/>
</p>

<o>

<kindRef ref="MusicProductKind"/>

</o>

<composedOf>

<statement

id="I-STATE-001"

layer="information"

type="state"

kind="AvailableMusicProductState">

<s>

<resource ref="Album42"/>

</s>

<p>

<predicate ref="inventoryState"/>

</p>

<o>

<value>InStock</value>

</o>

<composedOf>

```
<statement
  id="D-PROPERTY-001"
  layer="data"
  type="property"
  kind="MusicProductProperty">
```

```
<composedOf>
```

```
  <statement
    id="D-INSTANCE-001"
    layer="data"
    type="instance"
    source="music">
```

```
    <c ref="MusicCatalog"/>
```

```
    <s ref="Album42"/>
```

```
    <p ref="type"/>
```

```
    <o ref="Album"/>
```

```
  </statement>
```

```
<statement
  id="D-INSTANCE-002"
  layer="data"
  type="instance"
```

```

        source="store">

        <c ref="StoreCatalog"/>

        <s ref="Album42"/>

        <p ref="inventoryState"/>

        <o value="InStock"/>

    </statement>

</composedOf>

</statement>

</composedOf>

</statement>

</composedOf>

</statement>

</composedOf>

</statement>

```

This representation preserves the **semantic statement itself** and the **composition relationship through which it was produced**.

I. Statement Identity and Graph References

Although TreeGraph is naturally nested, the underlying semantic structure may be a graph rather than a strict tree because one lower-layer statement can contribute to multiple higher-layer statements.

For example:

(Album42, inventoryState, InStock)

could contribute simultaneously to:

AvailableMusicProductState

PurchasableProductState

MusicStorePurchaseRule

Consequently, a production representation should support references:

<composedOf>

 <statementRef ref="D-INSTANCE-002"/>

</composedOf>

rather than requiring every statement to be duplicated.

The underlying structure can therefore be understood as a **DAG serialized as a TreeGraph**, with stable statement IDs preserving semantic identity.

This is consistent with the document's emphasis on statement identity, recursive traversal, merging and context-preserving graph access.

J. Fold/Unfold Processing Metadata

The TreeGraph can additionally preserve how a statement was produced:

<fold

 id="FOLD-001"

 agent="AggregationAgent"

 operation="aggregate">

 <input ref="D-INSTANCE-001"/>

 <input ref="D-INSTANCE-002"/>

 <output ref="D-PROPERTY-001"/>

</fold>

followed by:

<fold

id="FOLD-002"

agent="AlignmentAgent"

operation="align">

<input ref="D-PROPERTY-001"/>

<output ref="I-RULE-001"/>

</fold>

and:

<fold

id="FOLD-003"

agent="ActivationAgent"

operation="activate">

<input ref="I-RULE-001"/>

<output ref="K-CONTEXT-001"/>

</fold>

This allows the representation to retain not only **what knowledge exists**, but also **which lower-layer statements contributed to it and which transformation produced it**.

It is particularly appropriate to the document's blackboard/stream-processing model, where agents merge incoming statements with already-known statements and publish the resulting augmented statements for further processing.

K. Layer Transformation Summary

The resulting transformation model is:

Layer	Statement types	Main operation	Output
Data	Instance → Type → Property	Aggregation / Folding	Structural semantic model
Information	Axis → State → Rule	Alignment / Folding	Dimensional/state/rule model
Knowledge	Actor → Role → Context	Activation / Folding	Executable semantic context
Reverse	Context → ... → Instance	Unfolding / De-aggregation	Source/composing statements

The forward path is:

CSPO

→ Type

→ Property

→ Axis

→ State

→ Rule

→ Actor

→ Role

→ Context

The reverse path is:

Context

→ Role

→ Actor / Rule

→ State / Axis

→ Property

→ Type

→ CSPO

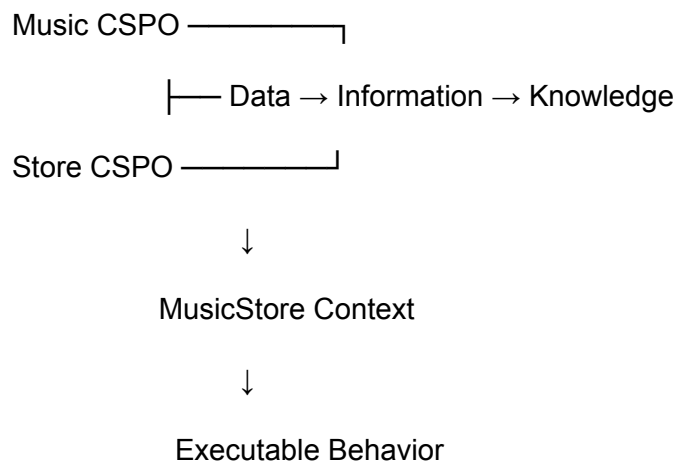
Each intermediate statement can therefore be treated simultaneously as:

1. **knowledge at its own layer,**
2. **input to the next layer,**
3. **a composition of lower-layer knowledge, and**
4. **a decomposition target for upstream processing.**

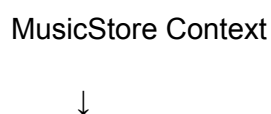
L. Architectural Principle

The combined model can consequently be summarized as:

FOLD / AGGREGATE



UNFOLD / DE-AGGREGATE



Information / Data



Source CSPO

The central principle is that **the executable Music Store model does not have to be a separately authored artifact**. It can emerge from the composition of independently sourced semantic statements, with each augmentation layer adding a new level of abstraction while retaining the lower-level statements that justify that abstraction.

The TreeGraph representation makes this layering explicit and makes the same message structure usable for both directions:

fold upward to infer and activate; unfold downward to explain, trace, transform, or recover source statements.

This is consistent with the document's existing description of lower-layer composition, upper-layer aggregation, upstream decomposition, and the use of TreeGraph/XSLT for nested statement processing.

Note: the exact XML vocabulary, identity-resolution rules, and reference/graph semantics proposed in this appendix are design elaborations of the document's TreeGraph and folding/unfolding concepts; they are not specified as a finalized XML schema in the source document.

Appendix — Stream-Oriented Folding and Unfolding of Layered Kinds

A. Purpose

This appendix extends the **Augmentation Layers Domain Model** with an operational view of how statements can be folded and unfolded while flowing through the layered streams.

The central idea is that **Merge, Fold, and Unfold are separate but cooperating operations**:

- **Merge** incorporates an incoming statement into the knowledge already established at its layer.
- **Fold** derives higher-layer statements from the newly merged lower-layer knowledge.
- **Unfold** recovers the composing lower-layer statements from a higher-layer statement.

This preserves the document's model in which statements are continuously merged with previously known statements, aggregated into higher-layer structures, and potentially decomposed back toward their source statements.

B. Layer Processing Model

The three augmentation layers are treated as streams:

Data

└─ Instance

└─ Type

└─ Property

Information

└─ Axis

└─ State

└─ Rule

Knowledge

└─ Actor

└─ Role

└─ Context

The resulting forward folding path is:

Data

↓

Information

↓

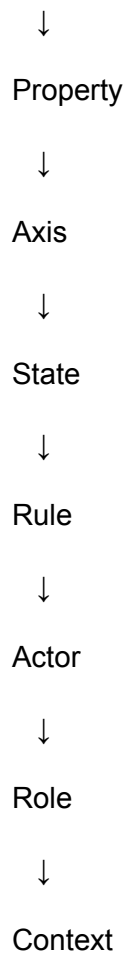
Knowledge

with the conceptual progression:

Instance

↓

Type



The document defines these statement families as the three augmentation layers, while describing the higher layers as increasingly abstract semantic structures derived from lower-layer statements.

C. Merge as the Central Stream Operation

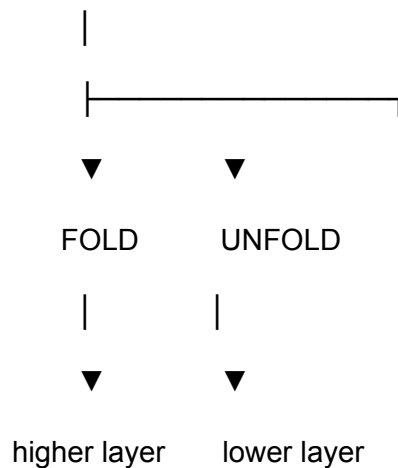
Each layer maintains a body of already-known statements.

An incoming statement is first merged with that state:

incoming statement



updated layer knowledge



Conceptually:

```
function MERGE(layer, statement):
```

```
    known = state[layer]
```

```
    merged = merge(known, statement)
```

```
    state[layer] = merged.state
```

```
    return merged.delta
```

The **delta** represents newly established knowledge that may justify further folding or unfolding.

This preserves the document's blackboard/stream-processing model, in which agents merge incoming statements with already-known statements and publish resulting augmented statements back into the processing stream.

D. Folding Kinds

Folding derives higher-level Kinds from the structural patterns observed in lower-level statements.

A conceptual operation is:

```
function FOLD_KINDS(statements):
```

```
    group statements by their
```

```
        Context
```

```
        Subject
```

```
        Predicate
```

```
        Object
```

```
    infer:
```

```
        Context Kinds
```

```
        Subject Kinds
```

```
        Predicate Kinds
```

```
        Object Kinds
```

```
    return inferred Kinds
```

This corresponds to the document's description of Kinds as functional extensions over the CSPO dimensions:

CK: Context $\rightarrow$ Subject $\rightarrow$ Predicate

SK: Subject $\rightarrow$ Predicate $\rightarrow$ Object

PK: Predicate $\rightarrow$ Subject $\rightarrow$ Object

OK: Object $\rightarrow$ Predicate $\rightarrow$ Subject

The important point is that folding is **incremental**. A higher-level Kind can become more complete as additional statements arrive in the stream.

E. Data-Layer Folding

The Data layer starts from concrete CSPO/instance statements.

For the Music Store example:

(MusicCatalog, Album42, type, Album)

(MusicCatalog, Album42, createdBy, Artist7)

(StoreCatalog, Album42, type, Product)

(StoreCatalog, Album42, inventoryState, InStock)

(StoreCatalog, Album42, availableIn, StoreA)

The Data stream can progressively fold these into structures such as:

AlbumKind

ProductKind

MusicProductKind

MusicProductProperty

Conceptually:

function processData(statement):

 delta = MERGE(Data, statement)

 kinds = FOLD_KINDS(delta)

 for kind in kinds:

 publish(Data, kind)

 for property in FOLD_DATA_PROPERTIES(delta):

 publish(Information, property)

Thus independently supplied Music and Store statements can progressively contribute to a common **Music Product** semantic structure.

The document's Music Store example similarly describes Music concepts such as Artist, Album, Track and Genre being composed with Store concepts such as Product, Inventory, Supplier, Order and Customer.

F. Information-Layer Folding

The Information layer consumes the Data-layer structures and aligns them into dimensions, states and rules.

Conceptually:

```
function processInformation(statement):
```

```
    delta = MERGE(Information, statement)
```

```
    axis = FOLD_AXIS(delta)
```

```
    for a in axis:
```

```
        publish(Information, a)
```

```
    states = FOLD_STATES(delta)
```

```
    for s in states:
```

```
        publish(Information, s)
```

```
    rules = FOLD_RULES(delta)
```

```
    for r in rules:
```

```
        publish(Information, r)
```

For example:

MusicProductProperty

+

InventoryProperty



The Information layer therefore transforms structural Data knowledge into **states and rules that describe meaningful situations and possible transitions**. This is consistent with the document's description of Alignment as transforming Property structures into Axis, State, Transition and Rule structures.

G. Knowledge-Layer Folding

The Knowledge layer consumes Information Rules and activates them into Actors, Roles and Contexts.

```
function processKnowledge(statement):
```

```
    delta = MERGE(Knowledge, statement)
```

```
    actors = FOLD_ACTORS(delta)
```

```
    for actor in actors:
```

```
        publish(Knowledge, actor)
```

```
    roles = FOLD_ROLES(delta)
```

```
    for role in roles:
```

```
publish(Knowledge, role)
```

```
contexts = FOLD_CONTEXTS(delta)
```

```
for context in contexts:
```

```
    publish(Knowledge, context)
```

For the Music Store:

MusicStorePurchaseRule

↓

CustomerActor

ProductActor

StoreActor

↓

CustomerRole

ProductRole

StoreRole

↓

MusicStorePurchaseContext

The Context can then provide the semantic basis for an executable interaction such as:

Purchase

├— CreateOrder

├— ReserveInventory

└— RecordPurchase

This follows the document's Activation concept, where Information-layer rules are turned into executable Knowledge-layer contexts and interactions.

H. Unfolding / De-aggregation

Folding has a corresponding reverse operation.

A higher-layer statement contains, explicitly or by reference, the statements from which it was composed.

For example:

MusicStorePurchaseContext

↓ unfold

MusicStorePurchaseRule

↓ unfold

AvailableMusicProductState

↓ unfold

MusicProductProperty

↓ unfold

MusicProductKind

↓ unfold

original CSPO statements

Conceptually:

function UNFOLD(statement):

if not hasComposition(statement):

return []

return compositionOf(statement)

However, the recovered statements should **not simply bypass the normal pipeline**.

Each unfolded statement is reintroduced into its own layer and merged there:

function propagateUnfolded(statement):

for child in UNFOLD(statement):

```
layer = layerOf(child)
```

```
delta = MERGE(  
    state[layer],  
    child  
)
```

```
state[layer] = delta.state
```

```
PROCESS_LAYER(  
    layer,  
    delta  
)
```

Consequently, unfolding becomes a recursive process of **de-aggregation followed by normal layer processing**.

The document explicitly describes the reverse operation as merging deeper aggregation statements and unfolding them until the raw S/P/O statements are reached.

I. Unified Layer Algorithm

The complete stream operation can therefore be expressed as:

```
function PROCESS(statement):
```

```
    layer = layerOf(statement)
```

```
    # 1. Merge into existing knowledge
```

```
    delta = MERGE(  
        state[layer],  
        statement  
)
```

```
    state[layer],  
    statement  
)
```

```
state[layer] = delta.state
```

2. Fold toward the next layer

```
for s in FOLD_LAYER(layer, delta):
```

```
    PUBLISH(  
        nextLayer(layer),  
        s  
    )
```

3. Unfold toward the previous layer

```
for s in UNFOLD_LAYER(layer, delta):
```

```
    PUBLISH(  
        previousLayer(layer),  
        s  
    )
```

With:

```
FOLD_LAYER(Data, ...)
```

→ Data Type / Property

→ Information

FOLD_LAYER(Information, ...)

→ Axis / State / Rule

→ Knowledge

FOLD_LAYER(Knowledge, ...)

→ Context / executable structures

and:

UNFOLD_LAYER(Knowledge, ...)

→ Information

UNFOLD_LAYER(Information, ...)

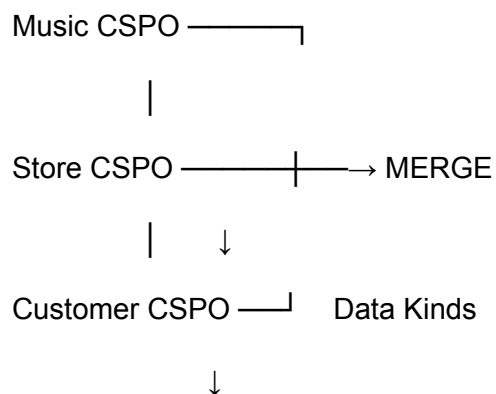
→ Data

UNFOLD_LAYER(Data, ...)

→ source CSPO

J. Music Store End-to-End Example

A simplified forward stream can therefore be represented as:



Data Properties



MERGE



Information Axis



Information State



Information Rule



MERGE



Knowledge Actor



Knowledge Role



MusicStore Context



Executable

Interaction

The reverse path is:

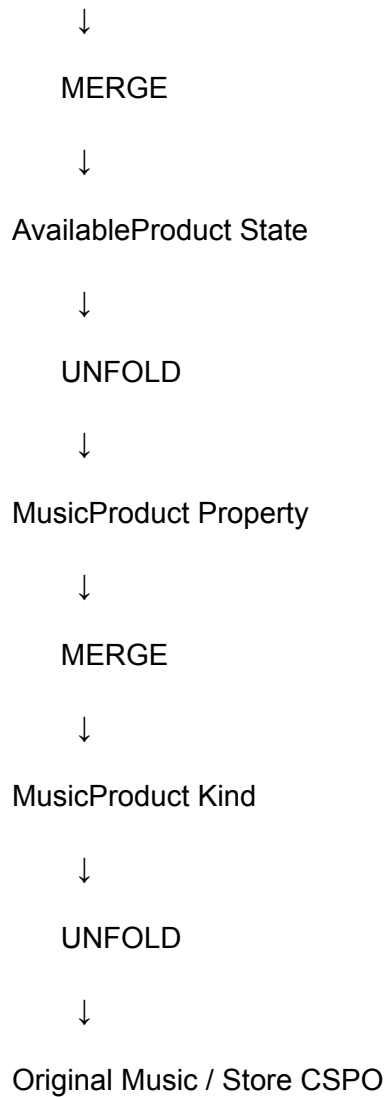
MusicStore Context



UNFOLD



MusicStore Rule



This makes the layered model **bidirectional** rather than simply a one-way hierarchy.

K. TreeGraph Representation

The folding/unfolding process can be represented in the proposed TreeGraph format using nested statements:

```
<statement  
  id="K-CONTEXT-001"  
  layer="knowledge"  
  type="context"  
  kind="MusicStorePurchaseContext">
```

<composedOf>

<statementRef ref="I-RULE-001"/>

</composedOf>

</statement>

The referenced Information Rule may contain:

<statement

id="I-RULE-001"

layer="information"

type="rule"

kind="MusicStorePurchaseRule">

<composedOf>

<statementRef ref="I-STATE-001"/>

</composedOf>

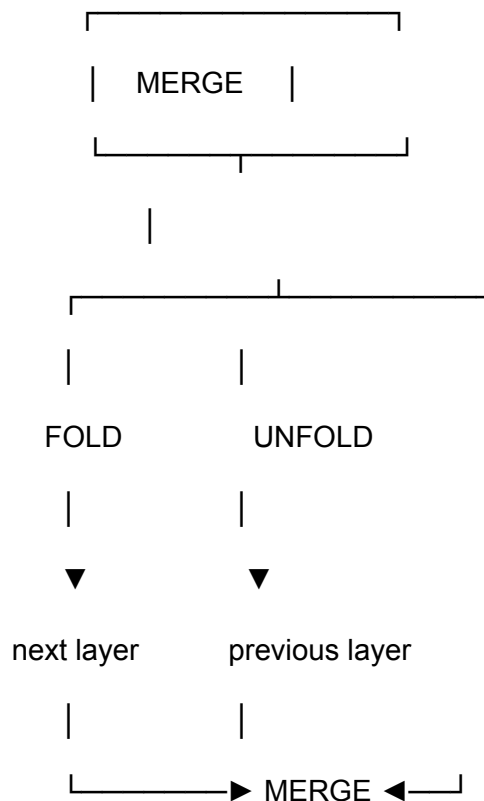
</statement>

and so on down to the original Data statements.

This representation should preserve **stable statement identity** through IDs/references, because one lower-layer statement may participate in multiple higher-layer folds. The document's TreeGraph proposal and emphasis on recursive traversal and identity support this interpretation.

L. Architectural Principle

The resulting operational model can be summarized as:



The essential invariant is:

Every folded statement is a semantic result that can be consumed at its own layer and can retain enough composition information to be unfolded back into its contributing statements. Every unfolded statement re-enters the ordinary merge/fold/unfold pipeline at its corresponding layer.

This makes **Merge** the stateful convergence mechanism, **Fold** the upward augmentation mechanism, and **Unfold** the downward de-aggregation mechanism.

Together they provide a stream-oriented implementation of the document's **Aggregation** → **Alignment** → **Activation** architecture while preserving the ability to move both downstream toward executable Knowledge and upstream toward the original Data/CSPO statements.

Appendix: ChatGPT Conversation

The key insight: behavior as inferred graph structure

Traditional ontology systems stop at:

Customer
owns
Order

Traditional applications add behavior manually:

```
customer.placeOrder()
```

Your proposal introduces a missing layer:

```
Customer
  asks_for
    Product
```

```
Product
  available_in
    Inventory
```

```
Inventory
  managed_by
    Store
```

From which the runtime infers:

```
PotentialContext:
  Sale
```

and then:

```
ExecutableInteraction:
  CreateOrder
  ReserveInventory
  GenerateInvoice
```

The behavior is not explicitly programmed.

It emerges from graph topology plus ontology definitions.

This is a profound shift.

DCI is surprisingly relevant

Most modern developers have never worked with DCI (Data, Context, Interaction), but your description maps almost perfectly to it.

The classic DCI idea was:

Data

```
Customer
Product
Order
```

Inventory

Context

SaleContext

Interaction

Customer buys Product
Inventory reserves Product
Order records transaction

The revolutionary aspect of DCI was:

Objects do not own behavior.

Contexts own behavior.

Your proposal essentially generalizes DCI into graph semantics.

Instead of:

SaleContext.execute()

you have:

Graph Pattern



Context Activation



Interaction Instances

Your pipeline resembles a semantic operating system

The stages you describe look something like:

Raw Triple Stream



Role Inference



Type Inference



Entity Resolution



Context Discovery



Interaction Discovery



Execution Planning

↓
Runtime Activation

which is very different from:

Source Code

↓
Compiler

↓
Binary

The graph itself becomes the executable substrate.

The music store example is particularly interesting

Suppose we have:

Music ontology

Artist
Album
Track
Genre

and:

Store ontology

Product
Inventory
Supplier
Order
Customer

Current software engineering would create:

MusicStoreApplication

by writing code.

Your system would instead perform semantic composition.

The runtime discovers:

Album
 subclassOf
 Product
Artist

actsAs
Supplier
Track
digitalVariantOf
Product

which yields:

MusicStoreDomain

without a developer explicitly coding it.

The application emerges from ontology alignment.

The crucial missing concept: affordances

I think your model needs one additional semantic layer.

The graph contains:

Customer
Product
Inventory

but what creates:

SaleContext

?

Not merely relationships.

What creates it is an affordance.

Something like:

Product
affords
Purchase
Inventory
affords
Reservation
Customer
affords
Ownership

The runtime reasons:

Purchase
+ Reservation
+ Ownership

⇒ SaleContext

This becomes similar to planning systems in AI.

AI becomes a graph completion engine

In current architectures:

AI
→
generates code

In your architecture:

AI
→
completes graph

For example:

Given:

Artist
Album
Track
Inventory
Customer

The AI infers:

DigitalDownload
License
Recommendation
Playlist
Subscription

as missing concepts.

Humans approve or reject them.

The graph evolves.

The runtime could be deterministic

This is the most important part.

Many people imagine AI-native systems as:

User
↓
LLM
↓
Magic

which is impossible to audit.

Your proposal keeps a deterministic core:

Inference Layer
↓
Graph
↓
Validation
↓
Executable Contexts

Once the graph is accepted:

Execution

is deterministic.

The AI is only involved in graph augmentation.

Not execution.

This is a critical distinction.

Why this may be more realistic now than in 2005

The Semantic Web community largely assumed:

Humans
 create ontologies
Humans
 maintain ontologies
Humans
 align ontologies

which turned out to be prohibitively expensive.

Today:

Humans
provide intent

AI
discovers mappings

AI
proposes contexts

AI
proposes interactions

Humans
approve

which is much more feasible.

The biggest unresolved challenge

I don't think the hardest problem is execution.

I think it's **behavioral ontology design**.

We already have mature ways to represent:

Artist
Album
Customer
Product

using semantic technologies.

What we lack is a universal ontology for:

Sell
Reserve
Purchase
Deliver
Authorize
Approve
Recommend
Subscribe

In other words:

The Semantic Web standardized nouns.

Your runtime would require standardizing verbs.

Or more precisely:

Standardizing context activation patterns and interaction semantics.

That is where DCI becomes valuable, because DCI is one of the few software models that treats interactions themselves as first-class entities.

My assessment is that if such a system were built successfully, the final artifact would not look like a programming language, a database, or an ontology. It would look more like an **executable semantic knowledge ecosystem**, where:

Intent



Knowledge Graph



Context Discovery



Interaction Activation



Deterministic Execution

and AI's role would be to continuously transform incomplete knowledge into executable knowledge while leaving the resulting execution model fully inspectable and auditable by humans. That is considerably closer to a "semantic operating system" than to a conventional software framework.

Now we're getting into territory that starts looking less like an application framework and more like a **computational semantics architecture**.

What I find interesting is that you're attempting to solve a problem that most semantic systems avoided:

How do we represent behavior without escaping into procedural code?

Historically, semantic systems represented facts:

(Customer123, purchases, Album456)

and programming languages represented behavior:

purchase(customer, album)

Your proposal is trying to represent both as graph structures.

The key distinction you're introducing

You are separating:

Behavioral schemas

Equivalent to DCI Context definitions, protocols, use cases.

Verb:

```
(  
  Purchase,  
  Customer.Available,  
  Purchase,  
  Product.InStock  
)
```

which means:

If

Customer in Available state

and

Product in InStock state

then

Purchase interaction is possible

This is not an execution.

It is a possibility.

A semantic affordance.

Behavioral instances

Actual occurrences.

Action:

```
(  
  Purchase#1234,  
  Customer:Bob,
```

Purchase,
Album:DarkSideOfTheMoon
)

This is equivalent to:

Interaction instance

or

Event occurrence

What I find elegant

You're implicitly creating three levels.

Ontology level

Customer
Product
Purchase

Context level

PurchaseContext

requires

Customer.Available
Product.InStock

Interaction level

Bob purchased Album42

This is remarkably similar to:

OWL
+
Planning Domain
+
Execution Trace

but unified into a single graph representation.

RDF Quad contexts become important

Traditional RDF triples:

S P O

cannot distinguish:

possible

from

actual

very well.

Quads introduce:

Context

S

P

O

which means:

PurchaseContext

Bob

purchases

Album42

can exist separately from:

InventoryContext

Album42

locatedIn

Warehouse7

This is where your DCI influence becomes visible.

The Quad Context becomes analogous to a DCI Context.

The recursive verb definition is interesting

You suggest something like:

Verb:

(

Verb,

```
    DomainActorTypeState,  
    Verb,  
    RangeActorTypeState  
)
```

which can recursively expand.

Example:

Purchase

expands into:

```
ReserveInventory  
CreateInvoice  
TransferOwnership  
ReduceStock
```

which further expands into:

```
SetAttribute  
AddLink  
RemoveLink  
TransformValue
```

Eventually reaching primitives.

This resembles a semantic version of:

Lambda Calculus

or

Microcode

for behavior.

Not unlike how CPUs reduce:

ADD

into lower-level operations.

You are proposing:

Sell

reduces into:

TransferOwnership

which reduces into:

RemoveLink

AddLink

which are primitive semantic operations.

This solves an old Semantic Web problem

The Semantic Web had excellent declarative knowledge representation.

It never found a satisfying answer for:

What happens?

after a fact becomes true.

Rule systems attempted this:

- SWRL
- RuleML
- production rules

but they always felt bolted on.

Your model instead treats behavior as ontology.

Not as an external rule engine.

Activation as graph pattern matching

I think the runtime you imagine is roughly:

Stage 1

Knowledge graph.

Bob

type Customer

Album42

type Product

Album42
state InStock

Stage 2

Verb discovery.

Purchase

requires

Customer.Available

Product.InStock

Stage 3

Pattern matching.

Bob matches Customer.Available

Album42 matches Product.InStock

Stage 4

Activation.

PurchaseAction#987

is created.

This is essentially:

Graph

→

Pattern Match

→

Context Activation

→

Interaction Instance

which is very elegant.

Primitive verbs as the semantic VM

This is the part that I think is the most important.

Eventually every behavior must become deterministic.

Suppose the primitive operations are:

AssignAttribute
RemoveAttribute
CreateResource
DeleteResource
CreateLink
RemoveLink
TransformValue
EmitEvent

Then:

Purchase

might compile to:

```
CreateLink(  
  Bob,  
  owns,  
  Album42  
)
```

```
RemoveLink(  
  Album42,  
  availableIn,  
  Inventory  
)
```

```
EmitEvent(  
  AlbumSold  
)
```

At this point the runtime becomes deterministic.

No AI involved.

Just graph transformations.

AI becomes an ontology synthesizer

The AI's role shifts dramatically.

Not:

Generate code.

Instead:

Discover verbs.

Discover contexts.

Propose activations.

Suggest mappings.

Then humans review.

Then the semantic runtime executes.

What this starts resembling

Strangely enough, your architecture is beginning to resemble a fusion of:

- RDF / OWL knowledge representation
- DCI behavioral modeling
- STRIPS / planning systems
- Event sourcing
- State machines
- Workflow engines
- WASM-like execution runtimes

all expressed through a common graph substrate.

The question I would investigate next

The crucial research question becomes:

Can every executable semantic behavior be reduced to a finite set of primitive graph transformation verbs?

If the answer is yes, then your runtime has something analogous to:

Machine Language

except the machine language is:

CreateNode
DeleteNode
CreateEdge
DeleteEdge
AssignValue
TransformValue
EmitEvent

and every higher-level Context, Verb, and Action ultimately compiles into those primitives.

If that reduction is possible, then what you're describing is no longer just an ontology system. It becomes a complete computational model where:

Knowledge
+
Behavior
+
Execution

are represented uniformly as graph structures, and application development becomes the iterative refinement of semantic models rather than the authoring of source code. That would be a fundamentally different software stack from the one we've been building since the 1970s.